НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Факультет програмних систем та прикладної математики
Кафедра прикладної математики

Курсова робота
Із дисципліни «Методи оптимізації»
на тему: Дослідження збіжності методу Розенброка
для безумовної та умовної оптимізації

Студента групи КМ-32
Мурскаєва Д. А.

Керівник:
ст. викладач, Ладогубець Т. С.
Кількість балів: _____
Оцінка: _____

Київ — 2026

ЗМІСТ

ВСТУП....3

1 ПОСТАНОВКА ЗАДАЧІ....4

2 ТЕОРЕТИЧНА ЧАСТИНА....5

2.1 Задача безумовної оптимізації та функція Розенброка....5

2.2 Метод Розенброка....5

2.3 Перебудова напрямків пошуку....7

2.4 Критерії завершення пошуку....9

2.5 Застосування методу Розенброка для умовної оптимізації....9

2.6 Методи одновимірного пошуку....9

3 ПРАКТИЧНА ЧАСТИНА....11

3.1 Дослідження збіжності методу....11

3.2 Дослідження умовної оптимізації....25

ВИСНОВКИ....37

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ....39

ДОДАТКИ....40

Додаток А....40

Додаток Б....49

ВСТУП

Методи чисельної оптимізації широко застосовуються для розв’язання задач у прикладній математиці, економіці, інженерії та машинному навчанні. Особливе значення мають методи нульового порядку, оскільки вони не потребують обчислення похідних цільової функції, що є суттєвою перевагою при роботі зі складними або аналітично невираженими функціями.

Метою курсової роботи є дослідження збіжності методу Розенброка при мінімізації функції Розенброка, а також аналіз впливу параметрів методу на кількість обчислень цільової функції та точність знаходження точки мінімуму.

Об’єктом дослідження є процес чисельної мінімізації функції багатьох змінних. Предметом дослідження є збіжність методу Розенброка залежно від початкового кроку, параметрів методу, критерію завершення пошуку та заданої точності.

У результаті виконання роботи реалізовано програмне забезпечення методу Розенброка, проведено дослідження збіжності для задач безумовної та умовної оптимізації, побудовано траєкторії пошуку мінімуму та проаналізовано вплив параметрів алгоритму на ефективність пошуку.

1 ПОСТАНОВКА ЗАДАЧІ

Дослідити збіжність методу Розенброка при мінімізації функції Розенброка в залежності від:

1. Значення початкового кроку $\|\Delta x\|$.
2. Параметрів методу α, β.
3. Вигляду критерію закінчення. $\left(\begin{cases} \frac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases} \text{ або } \|\Delta x\| \le \varepsilon \right)$.
4. Модифікацій методу.

Використати метод Розенброка в якості метода спуску для умовної оптимізації в залежності від:

1. Розташування локального мінімума (всередині / поза допустимою областю).
2. Виду допустимої області (опукла / невопукла / з лінійними обмеженнями).
3. Виду метода одновимірного пошуку (ДСК-Пауелла або Золотого перетину).
4. Точності метода одновимірного пошуку.
5. Значення параметру в алгоритмі Свена.

2 ТЕОРЕТИЧНА ЧАСТИНА

2.1 Задача безумовної оптимізації та функція Розенброка

Задача безумовної оптимізації полягає у знаходженні такої точки x^*, у якій цільова функція набуває мінімального значення:

$$minf\,(x)$$

де $x = \left[x_1, \ldots, x_n\right]$ – вектор змінних, $f\,(x)$ – цільова функція.

У роботі як тестова функція використовується функція Розенброка:

$$f\left(x_1, x_2\right) = 100\left(x_1^2 - x_2\right)^2 + \left(x_1 - 1\right)^2, \;\; x^{(0)} = (-1{,}2; 0) \tag{2.1}$$

Глобальний мінімум функції досягається в точці $x^* = (1; 1)$, де $f\,(x^*) = 0$. Особливістю функції Розенброка є наявність вузької криволінійної долини, що ускладнює пошук мінімуму та робить її зручною для тестування методів оптимізації.

2.2 Метод Розенброка

Метод Розенброка є методом нульового порядку, тобто не потребує обчислення похідних цільової функції. Пошук мінімуму виконується шляхом послідовного руху вздовж системи напрямків із подальшою перебудовою цих напрямків залежно від результатів пошуку.

У роботі використовується двовимірний випадок, початкова система напрямків має вигляд:

$$\begin{aligned} \hat{S}_1^{(0)} &= \left[1; 0\right]^T \\ \hat{S}_2^{(0)} &= \left[0; 1\right]^T \end{aligned} \tag{2.2}$$

Для кожного напрямку задається відповідний початковий крок:

$$\lambda = \left[\lambda_1; \lambda_2\right]^T \tag{2.3}$$

На кожному кроці виконується перевірка значення цільової функції в новій точці:

$$x^{k+1} = x^k + \lambda^k \hat{S}^k \tag{2.4}$$

Якщо значення цільової функції в новій точці зменшується, тобто $f\left(x^{k+1}\right) < f\left(x^k\right)$, то переміщення вважається успішним, поточна точка оновлюється за формулою (2.4), а відповідний крок збільшується:

$$\lambda_{k+1} = \alpha\lambda_k, \;\; \alpha > 1 \tag{2.5}$$

Якщо значення цільової функції не зменшується, тобто $f\left(x^{k+1}\right) \geq f\left(x^k\right)$, то переміщення вважається неуспішним, поточна точка не змінюється, а відповідний крок змінюється за правилом:

$$\lambda_{k+1} = \beta\lambda_k, \quad -1 < \beta < 0 \tag{2.6}$$

Оскільки параметр β є від'ємним, при неуспішному переміщенні не тільки зменшується абсолютна величина кроку, але й змінюється напрямок руху.

У внутрішньому циклі методу для кожного з двох напрямків фіксується, чи було хоча б одне успішне та хоча б одне неуспішне переміщення. Зовнішня ітерація завершується тоді, коли для обох напрямків уже були зафіксовані успішні та неуспішні переміщення, а остання перевірка в обох напрямках є неуспішною.

Після завершення внутрішнього циклу кроки повертаються до значень, які відповідають останнім невдалим переміщенням. Для початкового циклу використовується правило:

$$\lambda_{k+1} = \beta\lambda_k \tag{2.7}$$

Для наступних циклів використовується правило:

$$\lambda_{k+1} = \frac{\beta^2}{\alpha}\lambda_k \tag{2.8}$$

Це потрібно для того, щоб після перебудови напрямків пошук продовжувався із кроками, скоригованими відповідно до останнього успішного та неуспішного руху.

2.3 Перебудова напрямків пошуку

Після завершення внутрішнього циклу методу Розенброка виконується перебудова системи напрямків. Це дозволяє адаптувати подальший пошук до форми поверхні цільової функції.

Нехай x^{k-1} – точка на початку зовнішньої ітерації, а x^k – точка, отримана після завершення внутрішнього циклу. Для побудови нової системи напрямків використовуються вектори $A_1^{(k)}$ та $A_2^{(k)}$.

Перший та другий вектори визначаються наступним чином:

$$A_1^{(k)} = \lambda_1^{(k)}\hat{S}_1^{(k)} + \lambda_2^{(k)}\hat{S}_2^{(k)} \tag{2.9}$$

$$A_2^{(k)} = \lambda_2^{(k)}\hat{S}_2^{(k)} \tag{2.10}$$

Після цього перший новий напрямок отримується нормуванням вектора $A_1^{(k)}$:

$$\hat{S}_1^{(k+1)} = \frac{A_1^{(k)}}{\|A_1^{(k)}\|} \tag{2.11}$$

Для побудови другого напрямку з вектора $A_2^{(k)}$ віднімається його проєкція на $\hat{S}_1^{(k+1)}$:

$$\mathrm{B}_2^{(k)} = A_2^{(k)} - \left[\left(A_2^{(k)}\right)^T \hat{S}_1^{(k+1)}\right] \hat{S}_1^{(k+1)} \tag{2.12}$$

Після цього другий напрямок визначається нормуванням вектора $\mathrm{B}_2^{(k)}$:

$$\hat{S}_2^{(k+1)} = \frac{\mathrm{B}_2^{(k)}}{\|\mathrm{B}_2^{(k)}\|} \tag{2.13}$$

Якщо норма $\mathrm{B}_2^{(k)}$ є дуже малою, то другий напрямок задається як ортогональний до першого:

$$\hat{S}_2^{(k+1)} = \left(\hat{S}_{12}^{(k)}; \hat{S}_{11}^{(k)}\right) \tag{2.14}$$

Таким чином, після кожної зовнішньої ітерації метод формує нову ортонормовану систему напрямків $\mathrm{S_1}$, $\mathrm{S_2}$, що дозволяє ефективніше рухатися вздовж напрямку зменшення цільової функції.

2.4 Критерії завершення пошуку

Ітераційний процес завершується після досягнення заданої точності. У роботі розглядаються два критерії завершення:

$$\left(\begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases} \text{ або } \|\Delta x\| \le \varepsilon \right) \tag{2.15}$$

Перший критерій враховує зміну координат та зміну значення цільової функції, другий критерій базується лише на нормі зміни координат.

2.5 Застосування методу Розенброка для умовної оптимізації

Задача умовної оптимізації має вигляд:

$$\begin{gathered} minf\,(x) \\ h_i(x) = 0, \;\; i = 1,...,m \\ g_i(x) \ge 0, \;\; i = m+1,...,p \end{gathered}$$

Метод Розенброка може використовуватися як метод спуску при розв’язанні задач умовної оптимізації. Для цього задача з обмеженнями перетворюється на послідовність задач безумовної оптимізації. У роботі досліджується вплив розташування локального мінімуму, початкової точки та виду допустимої області на збіжність методу.

2.6 Методи одновимірного пошуку

Довжина кроку λ^k може визначатися як стала величина або через розв’язання задачі одновимірної мінімізації:

$$minf\left(x^k + \lambda^k S^k\right) \tag{2.16}$$

Процес розв'язку задачі (2) складається з двох етапів:

– знаходження інтервалу невизначеності, тобто інтервалу $\left[\lambda_{min}^k, \lambda_{max}^k\right]$ в якому знаходиться $\lambda_{on}^{(k)}$;
– пошук $\lambda_{on}^{(k)}$.

Для знаходження інтервалу невизначеності використовується алгоритм Свенна:

$$\lambda_{k+1} = \lambda_k \pm \Delta\lambda \cdot 2^k, \;\; k = 0,1,... \tag{2.17}$$

Довжина кроку $\Delta\lambda$ в алгоритмі Свенна визначається наступним чином:

$$\Delta\lambda = \alpha\left(\frac{\|x^{(k)}\|}{\|S^{(k)}\|}\right), \tag{2.18}$$

де параметр $\alpha = 0,1$, значення якого можна змінювати від 0,001 до 1.

Для пошуку оптимального значення λ у знайденому інтервалі будуть використовуватися метод золотого перетину та метод ДСК-Пауелла. Метод золотого перетину поступово звужує інтервал невизначеності, а метод ДСК-Пауелла використовує квадратичну апроксимацію функції.

3 ПРАКТИЧНА ЧАСТИНА

3.1 Дослідження збіжності методу

Дослідження було розділено на 2 етапи:

- Перший етап: пошук інтервалів для кожного параметра, у якому метод Розенброка робить найменше обчислень цільової функції.
- Другий етап: перебирання усіх можливих комібнацій значень параметрів із певним кроком, для того, щоб знайти найкращу комбінацію для кожного ε.

1) Дослідження відповідно до значення початкового кроку $\|\Delta x\|$.

Початкові умови:

$$\begin{cases} \Delta x = [0{,}01,\ 0{,}05,\ 0{,}1,\ 0{,}15,\ 0{,}25,\ 0{,}5,\ 0{,}75,\ 1,\ 1{,}5,\ 2,\ 2{,}5,\ 3] \\ \alpha = 3 \\ \beta = -0{,}5 \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases} \\ \varepsilon = 10^{-4} \end{cases}$$

Графічне представлення першого дослідження збіжності функції Розенброка відповідно до значення початкового кроку зображено на рис. 1.

Після проведення першого дослідження, було визначено 6 пар значень початкового кроку, близько до яких, найімовірніше, будуть знаходитися справжні параметри мінімізації функції з найменшою кількістю обчислень цільової функції. Відповідно цих пар буде проводитися наступне дослідження.

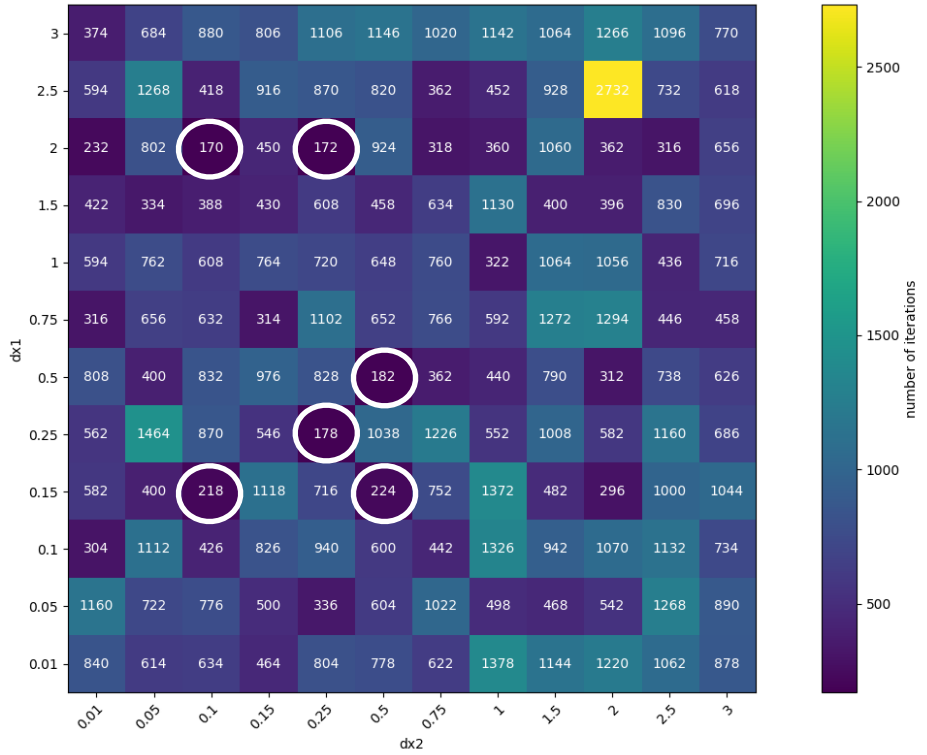


Рис. 1. Графічне представлення збіжності відповідно до початкового кроку

2) Дослідження відповідно до значення параметрів методу α, β.

Початкові умови:

$$\begin{cases} \Delta x = \left[\begin{pmatrix} 2 \\ 0{,}1 \end{pmatrix}, \begin{pmatrix} 2 \\ 0{,}25 \end{pmatrix}, \begin{pmatrix} 0{,}5 \\ 0{,}5 \end{pmatrix}, \begin{pmatrix} 0{,}25 \\ 0{,}25 \end{pmatrix}, \begin{pmatrix} 0{,}15 \\ 0{,}1 \end{pmatrix}, \begin{pmatrix} 0{,}15 \\ 0{,}5 \end{pmatrix} \right] \\ \alpha = [1{,}5,\ 2,\ 2{,}5,\ 3,\ 3{,}5,\ 4,\ 4{,}5,\ 5,\ 7,\ 10,\ 12,\ 15] \\ \beta = [-0{,}1, -0{,}15, -0{,}2, -0{,}25, -0{,}3, -0{,}35, -0{,}4, -0{,}5, -0{,}75, -0{,}9] \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases} \\ \varepsilon = 10^{-4} \end{cases}$$

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (2,\ 0{,}1)$ відповідно до значень параметрів зображено на рис. 2.



Рис. 2. Графічне представлення першого дослідження збіжності відповідно до значень параметрів

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (2,\ 0{,}25)$ відповідно до значень параметрів зображено на рис. 3.

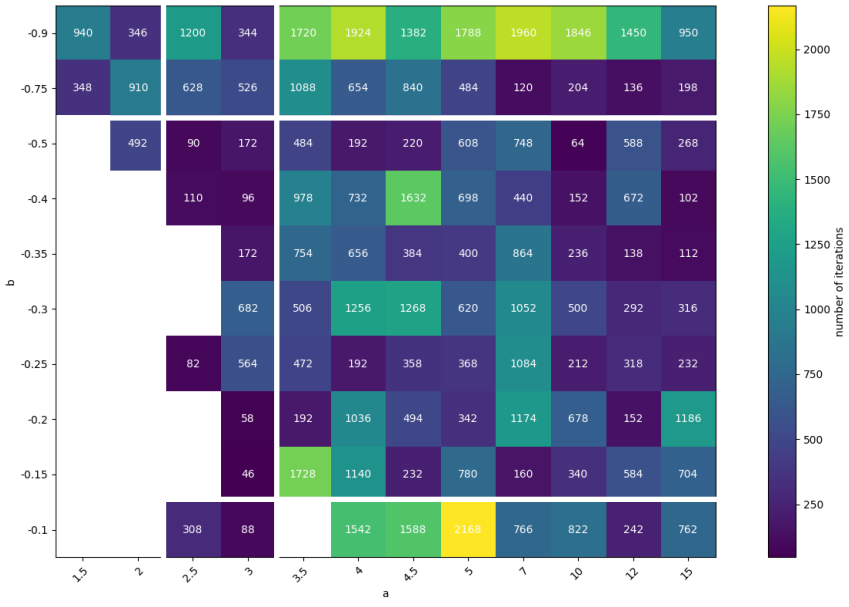


Рис. 3. Графічне представлення другого дослідження збіжності відповідно до значень параметрів

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (0{,}5,\ 0{,}5)$ відповідно до значень параметрів зображено на рис. 4.

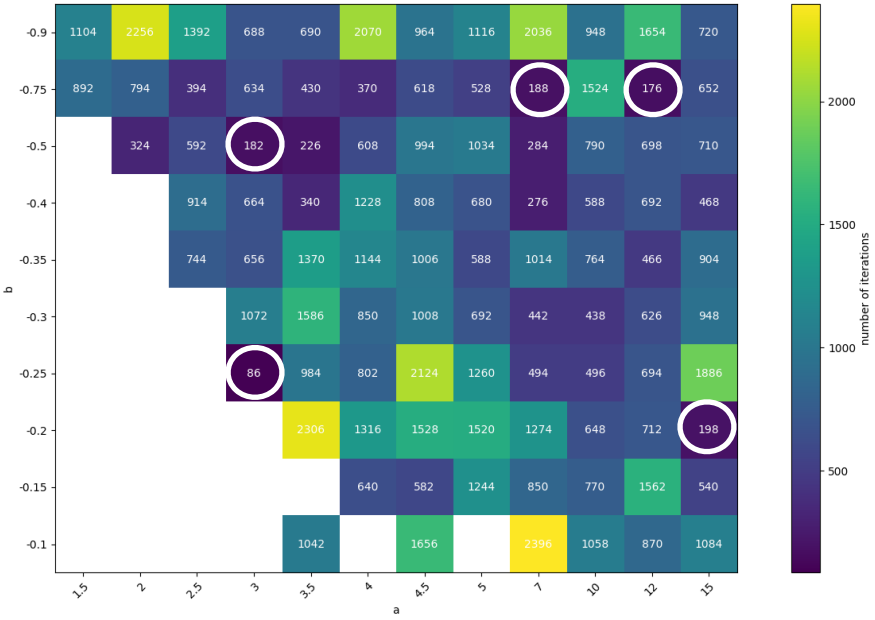


Рис. 4. Графічне представлення третього дослідження збіжності відповідно до значень параметрів

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (0{,}25,\ 0{,}25)$ відповідно до значень параметрів зображено на рис. 5.

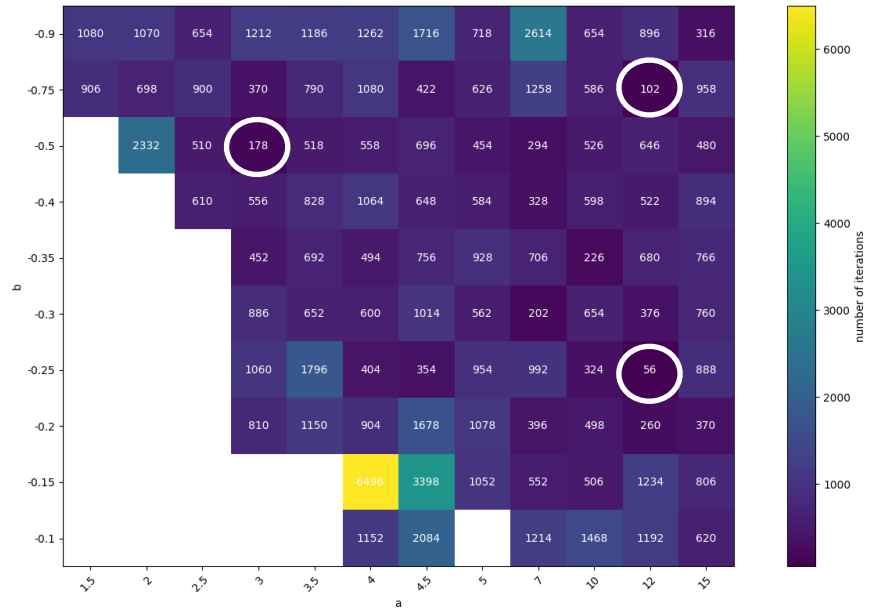


Рис. 5. Графічне представлення четвертого дослідження збіжності відповідно до значень параметрів

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (0{,}15,\ 0{,}1)$ відповідно до значень параметрів зображено на рис. 6.

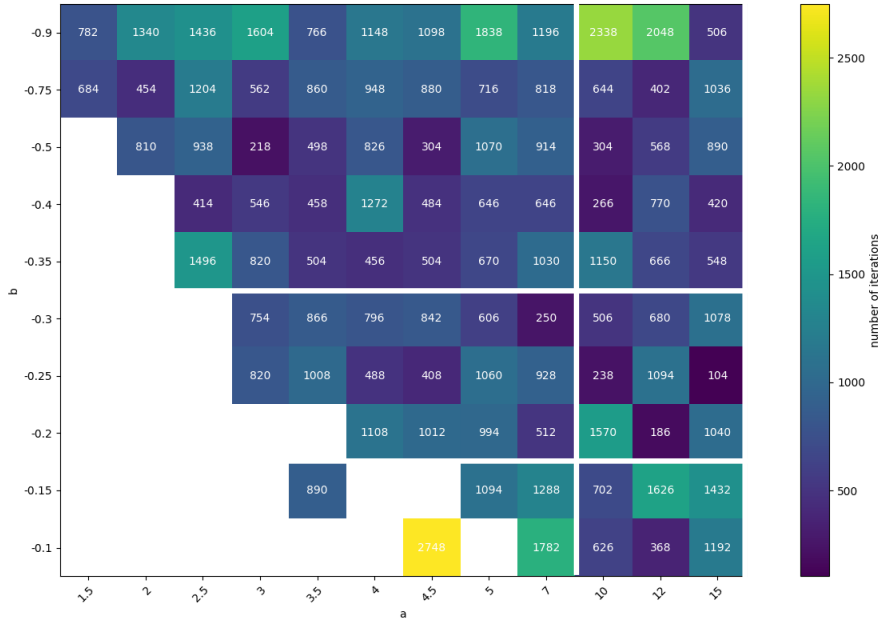


Рис. 6. Графічне представлення п’ятого дослідження збіжності відповідно до значень параметрів

Графічне представлення дослідження збіжності функції з початковими $\Delta x = (0{,}15,\ 0{,}5)$ відповідно до значень параметрів зображено на рис. 7.

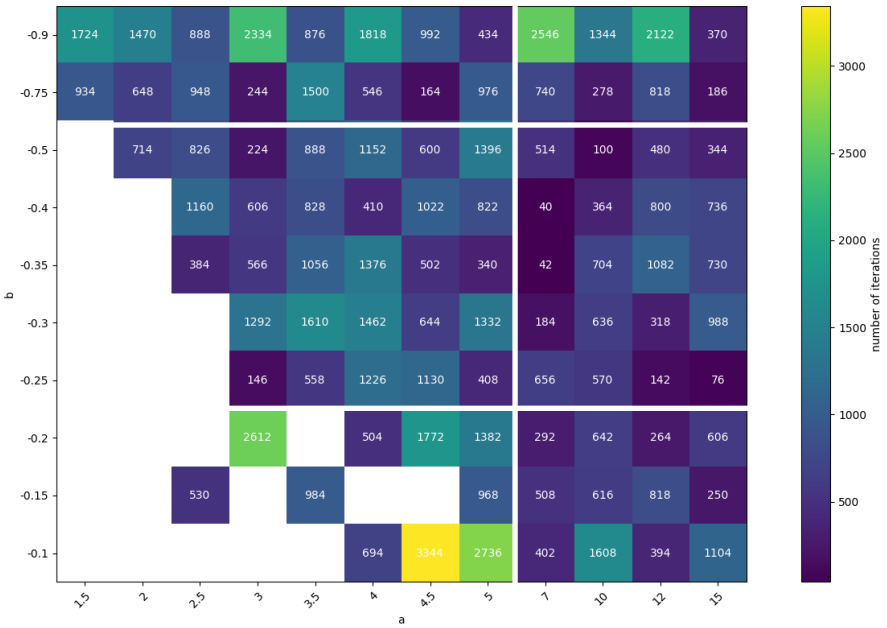


Рис. 7. Графічне представлення шостого дослідження збіжності відповідно до значень параметрів

Отже, було проведено ще додатково 6 досліджень на збіжність функції Розенброка відповідно до значень параметрів. На усіх попередніх рисунках було

виділено певні зони, для яких, із сталими початковими значеннями кроків, певні параметри показували значно кращі результати кількості обчислень цільової функції. Для фінального дослідження збіжності будуть обрані такі інтервали усіх параметрів, де функція збігалася за найменшу кількість кроків.

Перед тим, як проводити фінальне дослідження, потрібно перевірити правильність самої збіжності, оскільки алгоритм закінчував роботу при спрацюванні певного критерію закінчення, що не завжди свідчить про достатню збіжність функції до мінімуму.

Перевірка правильності мінімізації представлена у табл. 1, де для кожної пари значень початкового кроку були обрані найкращі параметри.

Таблиця 1. **Перевірка збіжності для** $\epsilon = 10^{-4}$

№	Вхідні дані		Вихідні дані			
	Δx	α, β	x_1, x_2	$f(x_1, x_2)$	n_r	n
1	$\begin{pmatrix} \Delta x_1 = 2 \\ \Delta x_2 = 0{,}1 \end{pmatrix}$	$\begin{cases} \alpha = 10 \\ \beta = -0{,}25 \end{cases}$	$\begin{cases} x_1 = 0{,}996968 \\ x_2 = 0{,}993725 \end{cases}$	0,000014	4	30
2	$\begin{pmatrix} \Delta x_1 = 2 \\ \Delta x_2 = 0{,}25 \end{pmatrix}$	$\begin{cases} \alpha = 3 \\ \beta = -0{,}15 \end{cases}$	$\begin{cases} x_1 = 0{,}994459 \\ x_2 = 0{,}988920 \end{cases}$	0,000031	4	46
3	$\begin{pmatrix} \Delta x_1 = 0{,}5 \\ \Delta x_2 = 0{,}5 \end{pmatrix}$	$\begin{cases} \alpha = 3 \\ \beta = -0{,}25 \end{cases}$	$\begin{cases} x_1 = 1{,}002480 \\ x_2 = 1{,}004930 \end{cases}$	0,000006	6	86
4	$\begin{pmatrix} \Delta x_1 = 0{,}25 \\ \Delta x_2 = 0{,}25 \end{pmatrix}$	$\begin{cases} \alpha = 12 \\ \beta = -0{,}25 \end{cases}$	$\begin{cases} x_1 = 1{,}753465 \\ x_2 = 3{,}076921 \end{cases}$	0,568230	7	56
5	$\begin{pmatrix} \Delta x_1 = 0{,}15 \\ \Delta x_2 = 0{,}1 \end{pmatrix}$	$\begin{cases} \alpha = 15 \\ \beta = -0{,}25 \end{cases}$	$\begin{cases} x_1 = 0{,}893606 \\ x_2 = 0{,}797884 \end{cases}$	0,011362	11	104
6	$\begin{pmatrix} \Delta x_1 = 0{,}15 \\ \Delta x_2 = 0{,}5 \end{pmatrix}$	$\begin{cases} \alpha = 7 \\ \beta = -0{,}4 \end{cases}$	$\begin{cases} x_1 = 0{,}993640 \\ x_2 = 0{,}987340 \end{cases}$	0,000040	5	40

З табл. 1 можна побачити, що функція не збігається достатньо у 2 випадках (4 і 5), хоча виконалися критерії закінчення. Обидві невдалі спроби мають найбільші значення α і найменшу суму початкових кроків $\|\Delta x\|$, відповідно до останньої вибірки. Отже, для фінального дослідження будуть використані $\alpha < 11$. Маленькі початкові кроки залишаться, оскільки існує вірогідність відкинути справжні пари. Тобто, фінальні інтервали будуть:

$$\begin{cases} \Delta x_1 \in [0{,}15,\ 2] \\ \Delta x_2 \in [0{,}05,\ 0{,}5] \\ \alpha \in [3,\ 11] \\ \beta \in [-0{,}4,\ -0{,}1] \end{cases}$$

Перед тим як переходити до фінального дослідження, потрібно дослідити критерії закінчення та ε для 2 найкращих та відносно різних вхідних параметрів, щоб підтвердити достовірність дослідження.

3) Дослідження відповідно до критерію закінчення.

Початкові умови:

$$\begin{cases} \Delta x = \left[\begin{pmatrix} 2 \\ 0{,}1 \end{pmatrix}, \begin{pmatrix} 0{,}15 \\ 0{,}5 \end{pmatrix}\right] \\ \alpha,\ \beta = \left[\begin{pmatrix} 10 \\ -0{,}25 \end{pmatrix}, \begin{pmatrix} 7 \\ -0{,}4 \end{pmatrix}\right] \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left|f\left(x^{k+1}\right) - f\left(x^k\right)\right| \le \varepsilon \end{cases} \text{ або } \|\Delta x\| \le \varepsilon \\ \varepsilon = 10^{-4} \end{cases}$$

Графічне представлення першого та другого дослідження збіжності функції Розенброка відповідно до критерію закінчення зображено на рис. 8.

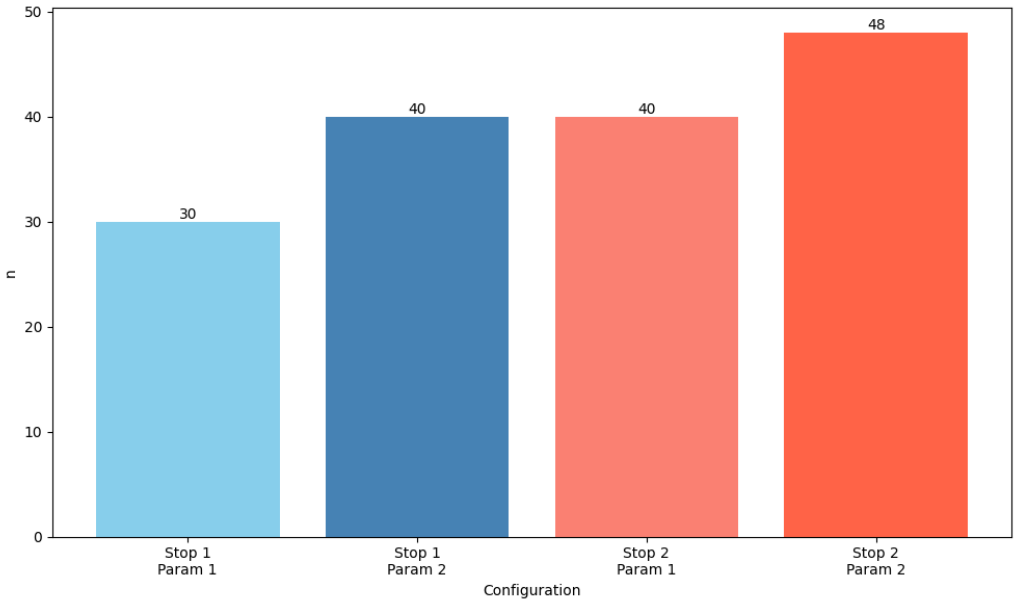


Рис. 8. Графічне представлення дослідження збіжності відповідно до значень параметрів та критерію закінчення

З рис. 8 Можна побачити, що в обох випадках використання першого критерію закінчення в методі буде робити менше обчислень цільової функції. Перевірка на правильність представлена у табл. 2.

Таблиця 2. **Перевірка критеріїв закінчення**

№	Вхідні дані			Вихідні дані		
	Δx	α, β	кр. зак.	x_1, x_2	$f(x_1, x_2)$	n
1	$\begin{pmatrix} \Delta x_1 = 2 \\ \Delta x_2 = 0{,}1 \end{pmatrix}$	$\begin{cases} \alpha = 10 \\ \beta = -0{,}25 \end{cases}$	1	$\begin{cases} x_1 = 0{,}996968 \\ x_2 = 0{,}993725 \end{cases}$	0,000014	30
			2	$\begin{cases} x_1 = 0{,}996913 \\ x_2 = 0{,}993788 \end{cases}$	0,000010	40
2	$\begin{pmatrix} \Delta x_1 = 0{,}15 \\ \Delta x_2 = 0{,}5 \end{pmatrix}$	$\begin{cases} \alpha = 7 \\ \beta = -0{,}4 \end{cases}$	1	$\begin{cases} x_1 = 0{,}993640 \\ x_2 = 0{,}987340 \end{cases}$	0,000040	40
			2	$\begin{cases} x_1 = 0{,}993671 \\ x_2 = 0{,}987340 \end{cases}$	0,000040	48

З табл. 2 Можна побачити, що за використання обох критеріїв закінчення результат буде схожий, але для першого критерію буде менше обчислень цільової функції.

4) Дослідження відповідно до значення ε.

Початкові умови:

$$\begin{cases} \Delta x_1 \in [0{,}15,\ 2] \\ \Delta x_2 \in [0{,}05,\ 0{,}5] \\ \alpha \in [3,\ 11] \\ \beta \in [-0{,}4, -0{,}1] \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases} \\ \varepsilon = [10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}, 10^{-7}, 10^{-8}, 10^{-9}, 10^{-10}, 10^{-11}] \end{cases}$$

Використовуючи отримані з попереднього етапу дослідження інтервали параметрів, переходимо до 2 етапу дослідження. За допомогою перебирання кожної комбінації параметрів з початкових умов, для кожного значення похибки були отримані параметри, за яких метод Розенброка робив мінімальну кількість обчислювано цільової функції. Результати отримані з цього етапу представлені у табл. 3. Відповідно значення n – кількість обчислень цільової функції, n_r – кількість змін напрямків методом Розенброка.

Таблиця 3. **Параметри з найменшою кількістю обчислень цільової функції для кожного ε**

Вхідні дані			Вихідні дані			
ε	Δx	α, β	x_1, x_2	$f(x_1, x_2)$	n_r	n
10^{-1}	$\begin{pmatrix}\Delta x_1 = 1{,}85\\ \Delta x_2 = 0{,}45\end{pmatrix}$	$\begin{cases}\alpha = 3\\ \beta = -0{,}1\end{cases}$	$\begin{cases}x_1 = 0{,}7007368\\ x_2 = 0{,}4762350\end{cases}$	0.111454	2	10
10^{-2}	$\begin{pmatrix}\Delta x_1 = 1{,}65\\ \Delta x_2 = 0{,}3\end{pmatrix}$	$\begin{cases}\alpha = 9{,}5\\ \beta = -0{,}1\end{cases}$	$\begin{cases}x_1 = 0{,}6003218\\ x_2 = 0{,}3571304\end{cases}$	0.160803	3	14
10^{-3}	$\begin{pmatrix}\Delta x_1 = 1{,}65\\ \Delta x_2 = 0{,}4\end{pmatrix}$	$\begin{cases}\alpha = 5{,}5\\ \beta = -0{,}2\end{cases}$	$\begin{cases}x_1 = 0{,}7656306\\ x_2 = 0{,}5862758\end{cases}$	0.054930	4	22
10^{-4}	$\begin{pmatrix}\Delta x_1 = 0{,}35\\ \Delta x_2 = 0{,}4\end{pmatrix}$	$\begin{cases}\alpha = 7{,}5\\ \beta = -0{,}3\end{cases}$	$\begin{cases}x_1 = 1{,}1463970\\ x_2 = 1{,}3149957\end{cases}$	0.021491	4	30
10^{-5}	$\begin{pmatrix}\Delta x_1 = 1{,}95\\ \Delta x_2 = 0{,}1\end{pmatrix}$	$\begin{cases}\alpha = 8\\ \beta = -0{,}4\end{cases}$	$\begin{cases}x_1 = 0{,}9936061\\ x_2 = 0{,}9872206\end{cases}$	0.000041	6	46
10^{-6}	$\begin{pmatrix}\Delta x_1 = 1{,}95\\ \Delta x_2 = 0{,}1\end{pmatrix}$	$\begin{cases}\alpha = 7\\ \beta = -0{,}15\end{cases}$	$\begin{cases}x_1 = 0{,}9998062\\ x_2 = 0{,}9996115\end{cases}$	3.76E-08	8	86
10^{-7}	$\begin{pmatrix}\Delta x_1 = 0{,}25\\ \Delta x_2 = 0{,}3\end{pmatrix}$	$\begin{cases}\alpha = 9{,}5\\ \beta = -0{,}4\end{cases}$	$\begin{cases}x_1 = 0{,}9999548\\ x_2 = 0{,}9999093\end{cases}$	2.05E-09	26	232
10^{-8}	$\begin{pmatrix}\Delta x_1 = 1{,}85\\ \Delta x_2 = 0{,}25\end{pmatrix}$	$\begin{cases}\alpha = 9{,}5\\ \beta = -0{,}4\end{cases}$	$\begin{cases}x_1 = 1{,}0000062\\ x_2 = 1{,}0000124\end{cases}$	3.81E-11	41	440
10^{-9}	$\begin{pmatrix}\Delta x_1 = 1{,}25\\ \Delta x_2 = 0{,}15\end{pmatrix}$	$\begin{cases}\alpha = 4{,}5\\ \beta = -0{,}2\end{cases}$	$\begin{cases}x_1 = 1{,}0000008\\ x_2 = 1.0000015\end{cases}$	5.79E-13	29	456
10^{-10}	$\begin{pmatrix}\Delta x_1 = 1{,}95\\ \Delta x_2 = 0{,}15\end{pmatrix}$	$\begin{cases}\alpha = 4{,}5\\ \beta = -0{,}25\end{cases}$	$\begin{cases}x_1 = 1{,}0000000\\ x_2 = 1{,}0000000\end{cases}$	4.41E-16	37	544
10^{-11}	$\begin{pmatrix}\Delta x_1 = 1{,}95\\ \Delta x_2 = 0{,}15\end{pmatrix}$	$\begin{cases}\alpha = 4{,}5\\ \beta = -0{,}25\end{cases}$	$\begin{cases}x_1 = 1{,}0000000\\ x_2 = 1{,}0000000\end{cases}$	1.37E-17	39	578

Графічне представлення дослідження збіжності функції Розенброка до значення $\varepsilon = 10^{-11}$ з наступними параметрами: $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$ та різними критеріями закінчення – зображено на рис. 9.

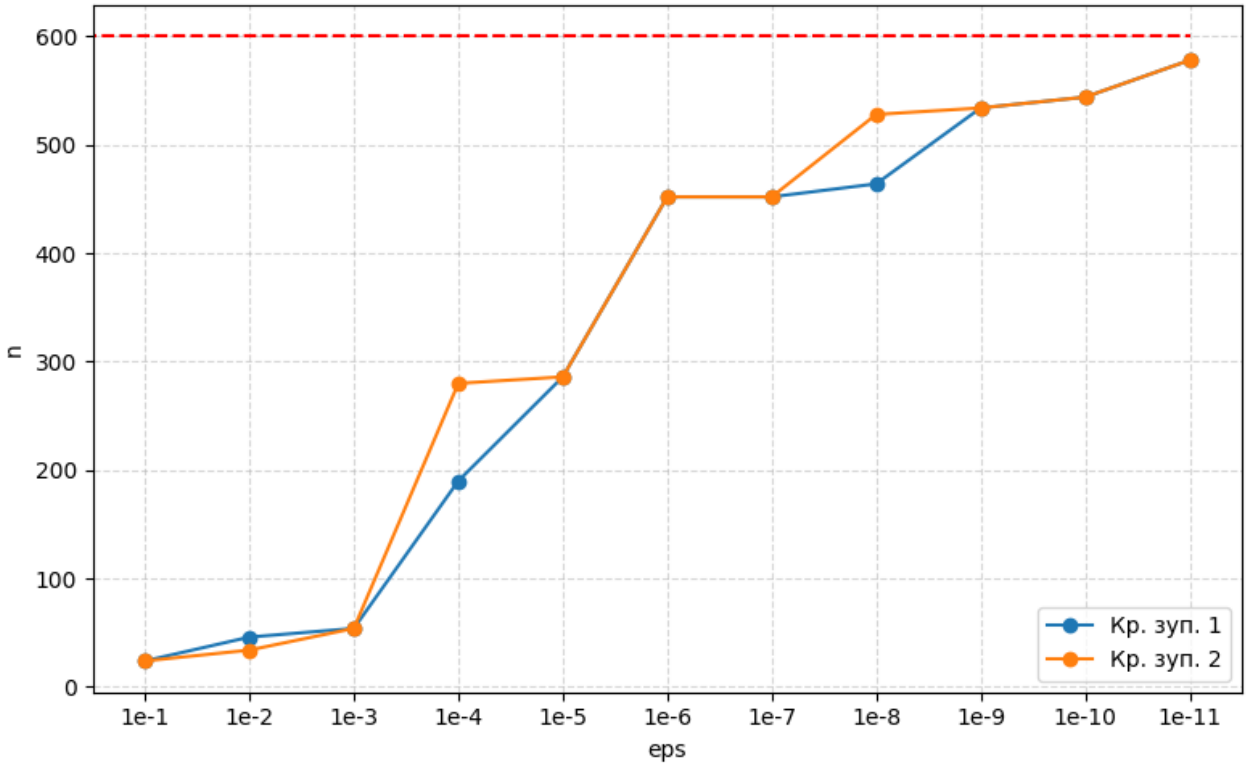


Рис. 9. Графічне представлення дослідження збіжності відповідно до значення ε

5) Дослідження відповідно до модифікацій методу.

Хоча алгоритм збігається до $\varepsilon = 10^{-11}$ за менше ніж 600 кроків при правильно підібраних параметрах, було досліджено певні модифікації методу. Зокрема, ділення чи множення параметрів на сталий коефіцієнт. Дійсно, такі модифікації можуть зменшити середню кількість обчислень цільової функції в обширному дослідженні. Проте, вони не приносять жодного результату для комбінацій параметрів, які є найкращими. Отже, комбінації представлені у табл. 3 є одними з найкращих для мінімізації. Вони не залежать від модифікацій, оскільки метод не «застрягає» через влучний підбір. Траєкторія мінімізації

функції Розенброка для $\varepsilon = 10^{-10}$ з наступними параметрами: $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$ зображено на рис. 10.

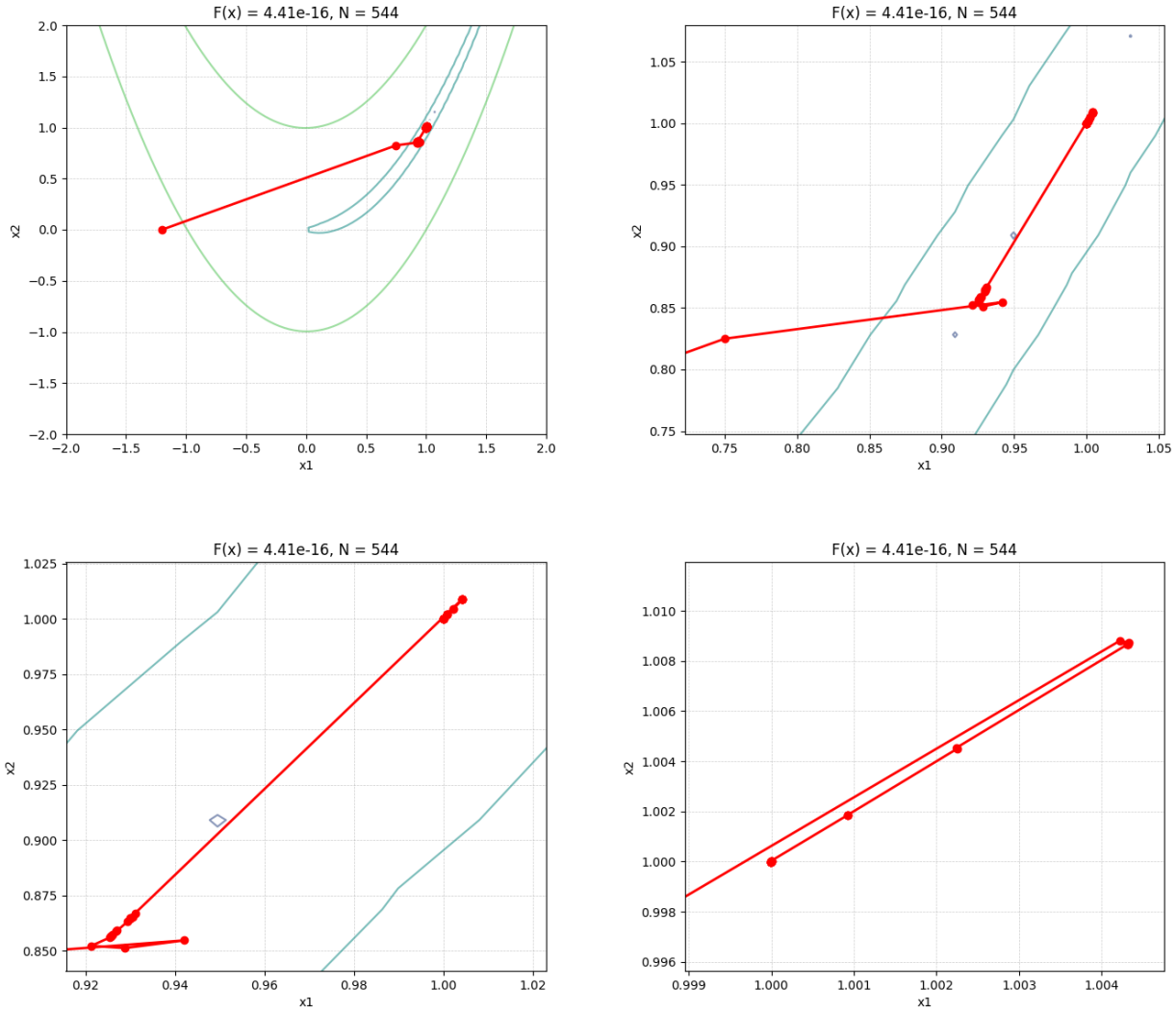


Рис. 10. Траєкторія мінімізації функції Розенброка для $\varepsilon = 10^{-10}$

Перевірка на правильність збіжності методу Розенброка з наступними параметрами: $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$ і різними критеріями закінчення представлена у табл. 4.

Таблиця 4. **Перевірка критеріїв закінчення для заданих параметрів**

ε	Критерій закінчення	x_1, x_2	$f(x_1, x_2)$	n_r	n
10^{-1}	1	$\begin{cases} x_1 = 0{,}9286991 \\ x_2 = 0{,}8512439 \end{cases}$	1.771E-02	3	24
	2	$\begin{cases} x_1 = 0{,}9286991 \\ x_2 = 0{,}8512439 \end{cases}$	1.771E-02	3	24
10^{-2}	1	$\begin{cases} x_1 = 0{,}9253706 \\ x_2 = 0{,}8560185 \end{cases}$	5.578E-03	5	46
	2	$\begin{cases} x_1 = 0{,}9212773 \\ x_2 = 0{,}8560185 \end{cases}$	7.450E-03	4	34
10^{-3}	1	$\begin{cases} x_1 = 0{,}9255057 \\ x_2 = 0{,}8566155 \end{cases}$	5.550E-03	6	54
	2	$\begin{cases} x_1 = 0{,}9255057 \\ x_2 = 0{,}8566155 \end{cases}$	5.550E-03	6	54
10^{-4}	1	$\begin{cases} x_1 = 0{,}9299643 \\ x_2 = 0{,}8646724 \end{cases}$	4.908E-03	14	190
	2	$\begin{cases} x_1 = 1{,}0043198 \\ x_2 = 0{,}8646724 \end{cases}$	1.869E-05	19	280
10^{-5}	1	$\begin{cases} x_1 = 1{,}0043175 \\ x_2 = 1{,}0086700 \end{cases}$	1.867E-05	20	286
	2	$\begin{cases} x_1 = 1{,}0043175 \\ x_2 = 1{,}0086700 \end{cases}$	1.867E-05	20	286
10^{-6}	1	$\begin{cases} x_1 = 0{,}9999983 \\ x_2 = 0{,}9999967 \end{cases}$	2.753E-12	30	452
	2	$\begin{cases} x_1 = 0{,}9999983 \\ x_2 = 0{,}9999967 \end{cases}$	2.753E-12	30	452

Продовження табл. 4

10^{-7}	1	$\begin{cases} x_1 = 0{,}9999983 \\ x_2 = 0{,}9999967 \end{cases}$	2.753E-12	30	452
10^{-7}	2	$\begin{cases} x_1 = 0{,}9999983 \\ x_2 = 0{,}9999967 \end{cases}$	2.753E-12	30	452
10^{-8}	1	$\begin{cases} x_1 = 0{,}9999983 \\ x_2 = 0{,}9999967 \end{cases}$	2.753E-12	31	452
	2	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 0{,}9999967 \end{cases}$	4.714E-16	35	528
10^{-9}	1	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	4.422E-16	36	534
	2	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	4.422E-16	36	534
10^{-10}	1	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	4.406E-16	37	544
	2	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	4.406E-16	37	544
10^{-11}	1	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	1.372E-17	39	578
	2	$\begin{cases} x_1 = 1{,}0000000 \\ x_2 = 1{,}0000000 \end{cases}$	1.372E-17	39	578

3.2 Дослідження умовної оптимізації

Для досліджень умовної оптимізації буде використовуватися інший критерій закінчення, зазначений у початкових умовах нижче, з $\varepsilon = 10^{-3}$.

1) Дослідження відповідно до розташування локального мінімума (всередині / поза допустимою областю).

Початкові умови:

$$\begin{cases} \Delta x = (1{,}95,\ 0{,}1) \\ \alpha = 4{,}5 \\ \beta = -0{,}25 \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \dfrac{\left|f\left(x^{k+1}\right) - f\left(x^k\right)\right|}{\left|f\left(x^k\right)\right|} \le \varepsilon \end{cases} \\ \varepsilon = 10^{-3} \\ \text{Опукла область 1: } x_1^2 + x_2^2 \le 2 \\ \text{Опукла область 2: } \left(x_1 + 0{,}5\right)^2 + x_2^2 \le 1 \end{cases}$$

Графічне представлення дослідження збіжності функції Розенброка відповідно до розташування локального мінімуму за кількістю обчислень цільової функції зображено на рис. 11.

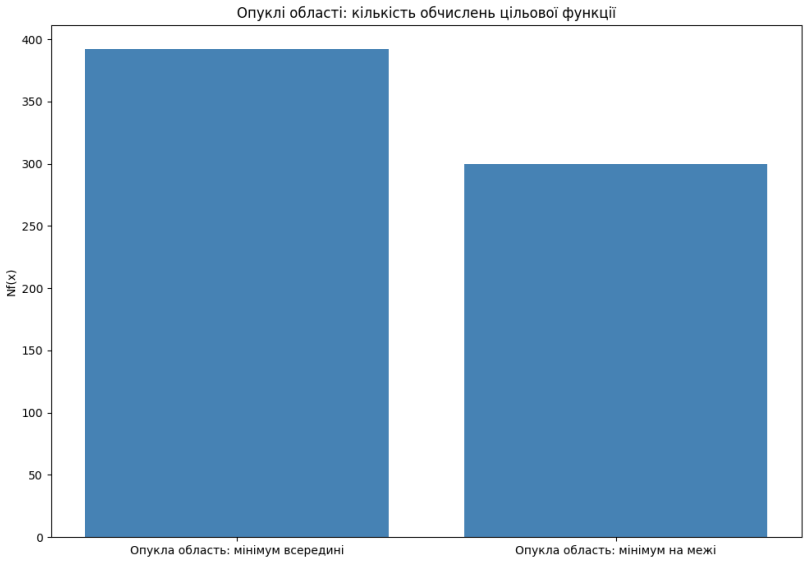


Рис. 11. Кількість обчислень цільової функції

Графічне представлення дослідження збіжності функції Розенброка відповідно до розташування локального мінімуму (всередині / поза допустимою областю) за траєкторією пошуку зображено на рис. 12.

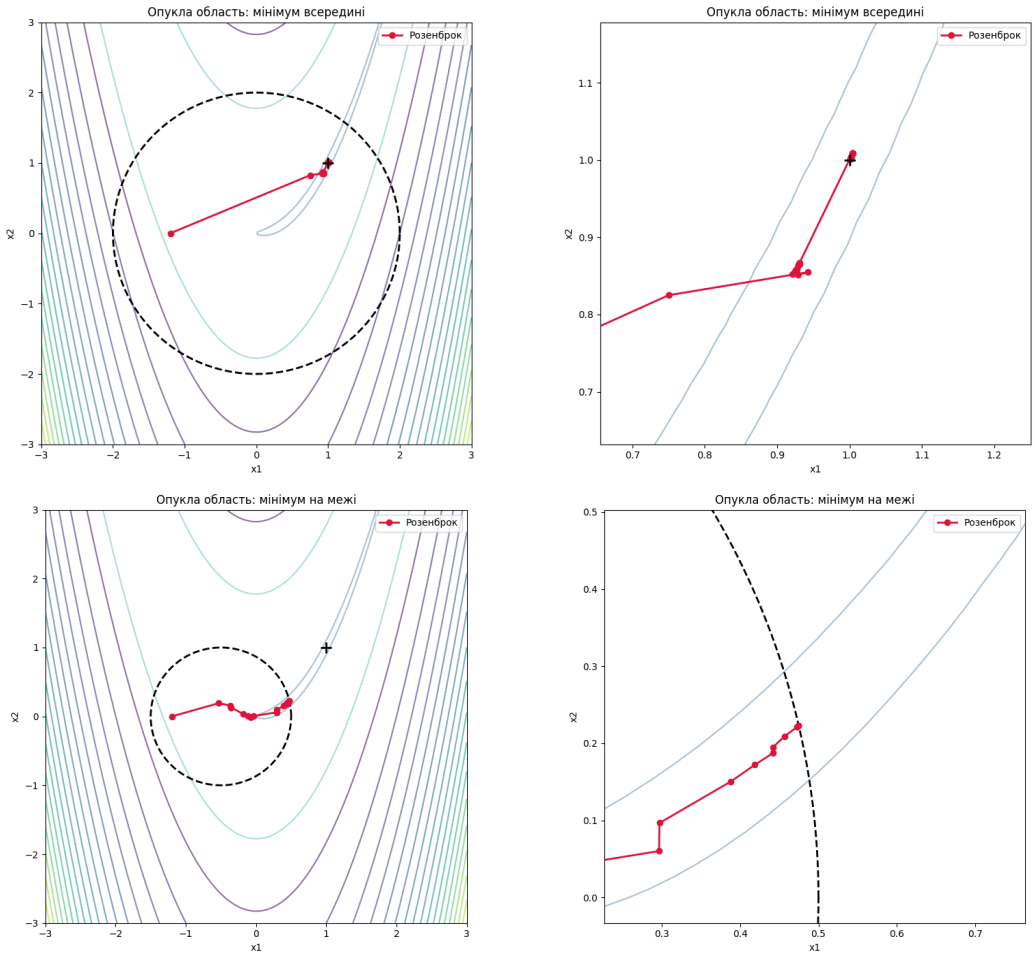


Рис. 12. Траєкторії пошуку локального мінімуму всередині / поза допустимою областю

Перевірка на правильність представлена у табл. 5.

Таблиця 5. **Перевірка збіжності всередині / поза допустимою областю**

Розташування мінімуму	n	x_1, x_2	Wolfram Alpha x_1, x_2	$f(x_1, x_2)$	Wolfram Alpha $f(x_1, x_2)$
Всередині області	392	$\begin{cases} x_1 = 1{,}0009227 \\ x_2 = 1{,}0018499 \end{cases}$	$\begin{cases} x_1 = 1 \\ x_2 = 1 \end{cases}$	0.0000009	0
Поза областю	300	$\begin{cases} x_1 = 0{,}4747097 \\ x_2 = 0{,}2230899 \end{cases}$	$\begin{cases} x_1 = 0{,}4747544 \\ x_2 = 0{,}2242 \end{cases}$	0.2764404	0.276203

З табл. 5 можна побачити, що пошук мінімуму, коли локальний мінімум розташований за межами допустимої області, потребує меншої кількості обчислень нових напрямків та обчислень цільової функції. Це пов’язано з тим, що тепер у задачах умовної оптимізації використовується новий критерій закінчення, який потребує значно кращої збіжності. Відповідно, методу Розенброка знадобиться менше ітерацій для того, щоб дійти до відповідної межі, ніж до самого мінімуму.

2) Дослідження відповідно до виду допустимої області (опукла / неопукла / з лінійними обмеженнями).

Початкові умови:

$$\begin{cases} \Delta x = \left(1{,}95,\ 0{,}1\right) \\ \alpha = 4{,}5 \\ \beta = -0{,}25 \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \dfrac{\left|f\left(x^{k+1}\right) - f\left(x^k\right)\right|}{\left|f\left(x^k\right)\right|} \le \varepsilon \end{cases} \\ \varepsilon = 10^{-3} \\ \text{Опукла область: } \left(x_1 + 0{,}5\right)^2 + x_2^2 \le 1 \\ \text{Лінійна область: } x_1 + x_2 \le 1.5 \\ \text{Неопукла область: } x_1^2 + x_2^2 \le 4,\ (x_1 - 1)^2 + (x_2 - 1)^2 \ge 0{,}1 \end{cases}$$

Опукла область була представлена у минулому дослідженні. У табл. 6 зображені остаточні локальні мінімуми для області певного виду. Важливо зазначити, що мінімуми не можна порівнювати, оскільки обмеження не однакові.

Таблиця 6. **Перевірка збіжності для різних видів допустимої області**

Вид області	n	x_1, x_2	Wolfram Alpha x_1, x_2	$f(x_1, x_2)$	Wolfram Alpha $f(x_1, x_2)$
Опукла	300	$\begin{cases} x_1 = 0{,}4747097 \\ x_2 = 0{,}2230899 \end{cases}$	$\begin{cases} x_1 = 0{,}4747544 \\ x_2 = 0{,}2242 \end{cases}$	0.2764404	0.276203
Лінійна	418	$\begin{cases} x_1 = 0{,}8230558 \\ x_2 = 0{,}6768997 \end{cases}$	$\begin{cases} x_1 = 0{,}823128 \\ x_2 = 0{,}676872 \end{cases}$	0.0313364	0.0313283
Неопукла	540	$\begin{cases} x_1 = 0{,}8498835 \\ x_2 = 0{,}7216746 \end{cases}$	$\begin{cases} x_1 = 1{,}134 \\ x_2 = 1{,}28644 \end{cases}$	0.0225743	0.0179978

Графічне представлення дослідження збіжності функції Розенброка відповідно до виду допустимої області (неопукла / з лінійними обмеженнями)

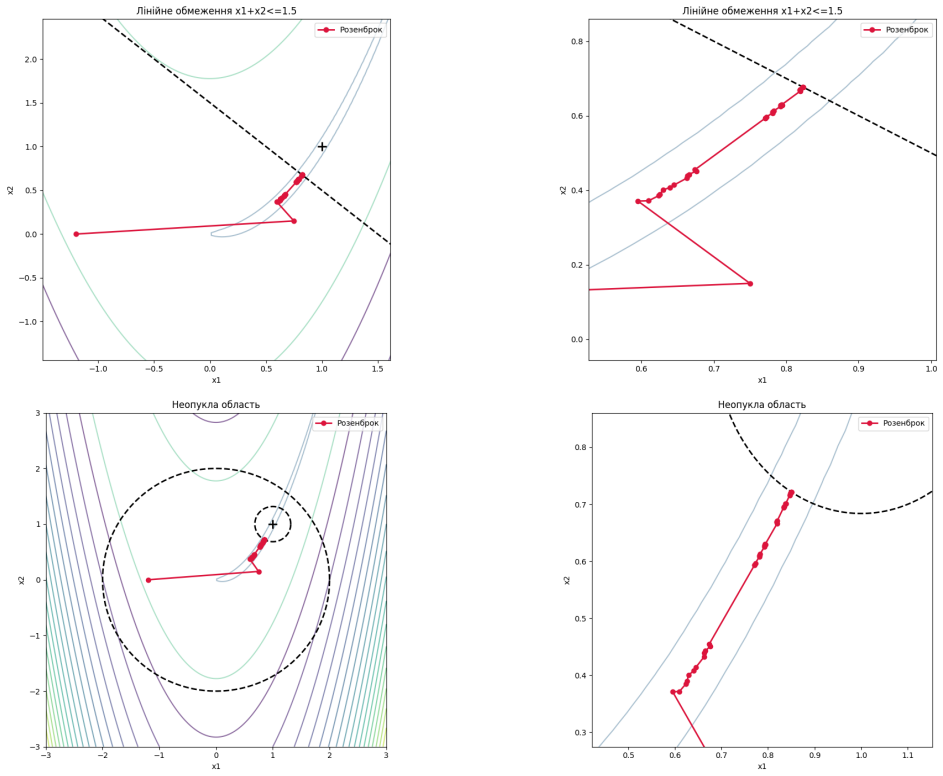


Рис. 13. Траєкторії пошуку локального мінімума відповідно до виду допустимої області

за траєкторією пошуку зображено на рис. 13.

Найкращий результат, відповідно до табл. 6, отримано для опуклої допустимої області. Лінійні обмеження збільшили кількість обчислень цільової функції до 418 разів. Неопукла область виявилася найскладнішою для

мінімізації, оскільки її геометрія ускладнює траєкторію руху методу, а наявність забороненої зони поблизу точки безумовного мінімуму змушує алгоритм перебудовувати напрямки пошуку. Важливо зазначити, що мінімум для неопуклої області відрізняється від обчисленого Wolfram Alpha, що може свідчити про складність виконання методу для неопуклих областей певною початковою точкою.

3) Дослідження відповідно до виду метода одновимірного пошуку (ДСК-Пауелла або Золотого перетину).

Початкові умови:

$$\begin{cases} \Delta x = (1{,}95,\ 0{,}15) \\ \alpha = 4{,}5 \\ \beta = -0{,}25 \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \dfrac{\left|f\left(x^{k+1}\right) - f\left(x^k\right)\right|}{\left|f\left(x^k\right)\right|} \le \varepsilon \end{cases} \\ \varepsilon = 10^{-3} \\ \text{метод одновимірного пошуку: ДСК-Пауелл, Золотий перетин} \\ \varepsilon_{1d} = 10^{-3} \\ \alpha_{sv} = 0.1 \\ \text{Лінійна область: } x_1 + x_2 \le 1.5 \end{cases}$$

Дослідження збіжності функції Розенброка відповідно до виду метода одновимірного пошуку показано у табл. 7 для лінійної області.

Таблиця 7. **Перевірка збіжності для лінійної області відповідно до метода одновимірного пошуку**

Вид методу	n	n_{1d}	x_1, x_2	$f\left(x_1, x_2\right)$	Wolfram Alpha $f\left(x_1, x_2\right)$
Золотий перетин	435	17	$\begin{cases} x_1 = 0{,}8230747 \\ x_2 = 0{,}6769253 \end{cases}$	0.0313303	0.0313283
ДСК-Пауелл	426	8	$\begin{cases} x_1 = 0{,}8230786 \\ x_2 = 0{,}6769214 \end{cases}$	0.0313300	0.0313283

З табл. 7 можна побачити, що використання методу одновимірного пошуку ДСК-Пауелла є значно кращим з точки зору кількості загальних обчислень. Також, обидва методи мінімізують функцію до одних і тих самих значень x_1, x_2. Отже, для розглянутих початкових умов метод ДСК-Пауелла виявився ефективнішим за кількістю обчислень, тоді як метод золотого перетину забезпечив подібну точність, але потребував на 9 більше обчислень. Траєкторії одновимірних пошуків представлені на рис. 14.

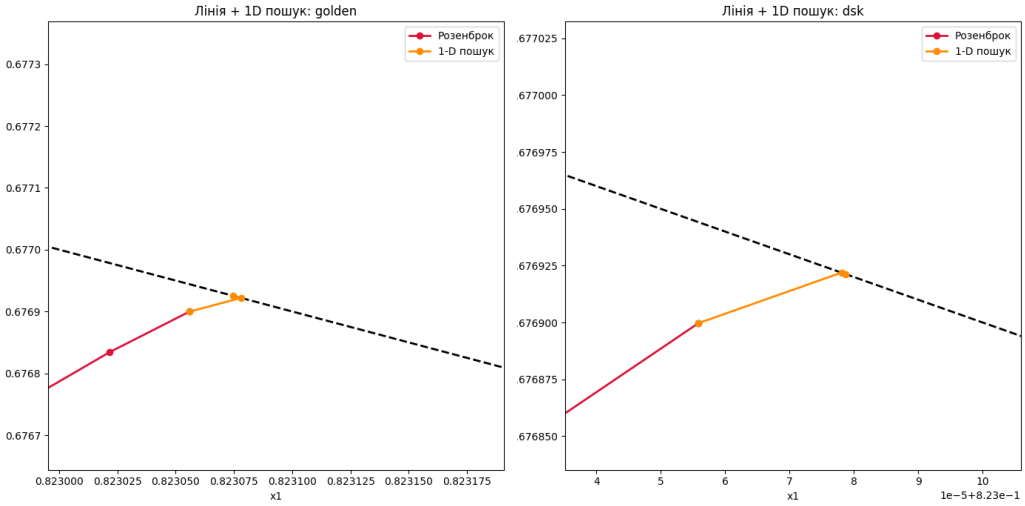


Рис. 14. Траєкторії пошуку локального мінімуму відповідно до виду одновимірного пошуку

4) Дослідження відповідно до точності метода одновимірного пошуку.

Початкові умови:

$$
\begin{cases}
\Delta x = (1{,}95,\ 0{,}15) \\
\alpha = 4{,}5 \\
\beta = -\,0{,}25 \\
\text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \dfrac{\left|f\left(x^{k+1}\right) - f\left(x^k\right)\right|}{\left|f\left(x^k\right)\right|} \le \varepsilon \end{cases} \\
\varepsilon = 10^{-3} \\
\text{метод одновимірного пошуку: ДСК-Пауелл, Золотий перетин} \\
\varepsilon_{1d} = \left[10^{-1},\ 10^{-2},\ 10^{-3},\ 10^{-4},\ 10^{-5},\ 10^{-6},\ 10^{-7},\ 10^{-8},\ 10^{-9},\ 10^{-10}\right] \\
\alpha_{sv} = 0.1 \\
\text{Лінійне обмеження: } x_1 + x_2 \le 1.5
\end{cases}
$$

Графічне представлення дослідження збіжності функції Розенброка відповідно до точності метода одновимірного пошуку Золотого перетину зображено на рис. 15.

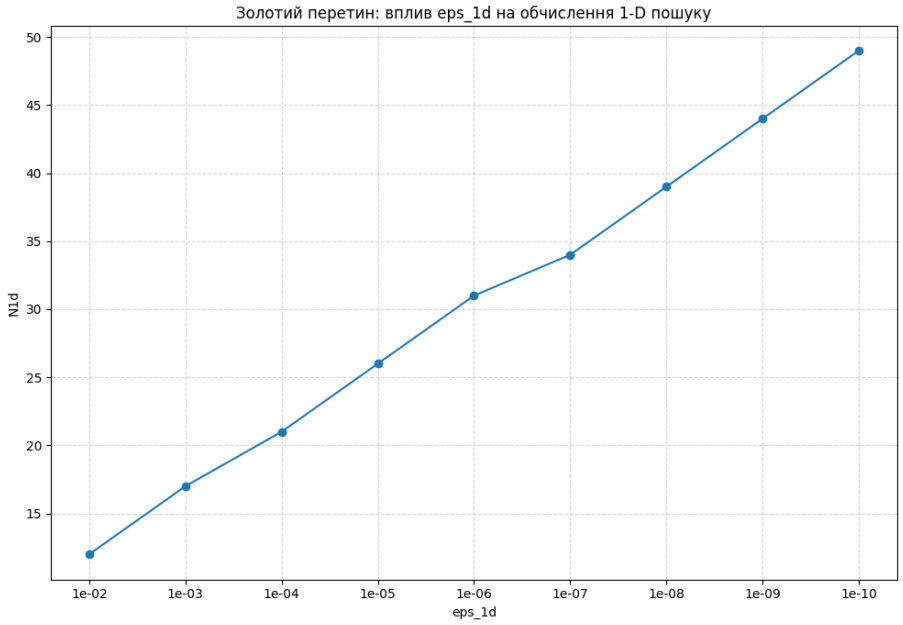


Рис. 15. Кількість обчислень цільової функції з методом Золотого перетину

Графічне представлення дослідження збіжності функції Розенброка відповідно до точності метода одновимірного пошуку ДСК-Пауелла зображено на рис. 16.

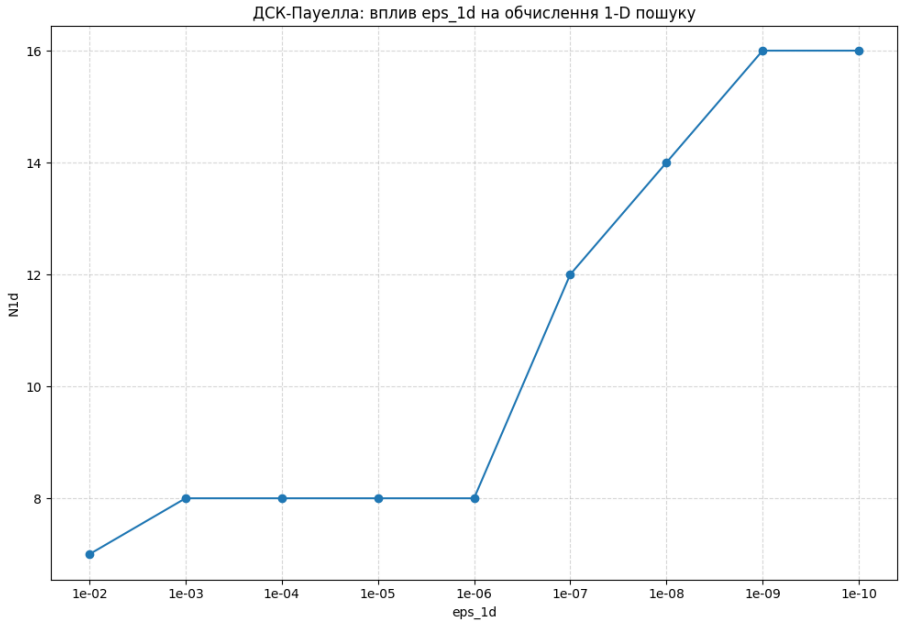


Рис. 16. Кількість обчислень цільової функції з методом ДСК-Пауелла

Як можна побачити з табл. 8, де представлена перевірка збіжності для методу ДСК-Пауелла та Золотого перетину, якість рішення значно покращується і практично стабілізується для $\varepsilon = 10^{-4}$. Ідентична ситуація виходить і для методу Золотого перетину з більшою кількістю обчислень (рис. 15). Хоча для однакової якості кінцевого розв'язку ДСК-Пауелл в цій задачі виявився економнішим за кількістю обчислень цільової функції, ніж золотий перетин для меншої похибки. Дослідження показало, що зменшення похибки не приводить до необмеженого покращення результату: після певного рівня точності координати розв'язку та значення цільової функції практично не змінюються, тоді як кількість обчислень цільової функції тільки зростає. Оптимальною слід вважати помірну точність одновимірного пошуку, а не максимально можливу.

Таблиця 8. **Перевірка збіжності для лінійно-обмеженої області пошуку використовуючи методи ДСК-Пауелла і Золотого перетину**

ε	Метод 1Д	x_1, x_2	$f(x_1, x_2)$	n_{1d}
10^{-2}	Золотий перетин	$\begin{cases} x_1 = 0.8245804 \\ x_2 = 0.6754196 \end{cases}$	0.0328090	12
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8212430 \\ x_2 = 0.6787570 \end{cases}$	0.0338176	7
10^{-3}	Золотий перетин	$\begin{cases} x_1 = 0.8230747 \\ x_2 = 0.6769253 \end{cases}$	0.0313303	17
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8230786 \\ x_2 = 0.6769214 \end{cases}$	0.0313300	8

Продовження табл. 8

10^{-4}	Золотий перетин	$\begin{cases} x_1 = 0.8231191 \\ x_2 = 0.6768809 \end{cases}$	0.0313283	21
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8230786 \\ x_2 = 0.6769214 \end{cases}$	0.0313300	8
10^{-5}	Золотий перетин	$\begin{cases} x_1 = 0.8231292 \\ x_2 = 0.6768708 \end{cases}$	0.0313283	26
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8230786 \\ x_2 = 0.6769214 \end{cases}$	0.0313300	8
10^{-6}	Золотий перетин	$\begin{cases} x_1 = 0.8231282 \\ x_2 = 0.6768718 \end{cases}$	0.0313283	31
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8230786 \\ x_2 = 0.6769214 \end{cases}$	0.0313300	8
10^{-7}	Золотий перетин	$\begin{cases} x_1 = 0.8231284 \\ x_2 = 0.6768716 \end{cases}$	0.0313283	34
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8231282 \\ x_2 = 0.6768718 \end{cases}$	0.0313283	12
10^{-8}	Золотий перетин	$\begin{cases} x_1 = 0.8231284 \\ x_2 = 0.6768716 \end{cases}$	0.0313283	39
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8231282 \\ x_2 = 0.6768718 \end{cases}$	0.0313283	14
10^{-9}	Золотий перетин	$\begin{cases} x_1 = 0.8231284 \\ x_2 = 0.6768716 \end{cases}$	0.0313283	44
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8231282 \\ x_2 = 0.6768718 \end{cases}$	0.0313283	16
10^{-10}	Золотий перетин	$\begin{cases} x_1 = 0.8231284 \\ x_2 = 0.6768716 \end{cases}$	0.0313283	49
	ДСК-Пауелл	$\begin{cases} x_1 = 0.8231282 \\ x_2 = 0.6768718 \end{cases}$	0.0313283	16

5) Дослідження відповідно до значення параметру в алгоритмі Свена.

Початкові умови:

$$\begin{cases} \Delta x = (1{,}95,\ 0{,}15) \\ \alpha = 4{,}5 \\ \beta = -0{,}25 \\ \text{кр. зак.} = \begin{cases} \dfrac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \dfrac{\left|f\left(x^{k+1}\right) - f\left(x^k\right)\right|}{\left|f\left(x^k\right)\right|} \le \varepsilon \end{cases} \\ \varepsilon = 10^{-3} \\ \text{метод одновимірного пошуку: ДСК-Пауелл} \\ \varepsilon_{1d} = 10^{-3} \\ \alpha_{sv} = [0{,}001,\ 0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1,\ 0{,}25,\ 0{,}5,\ 1] \\ \text{Лінійна область: } x_1 + x_2 \le 1.5 \end{cases}$$

Графічне представлення дослідження збіжності функції Розенброка відповідно до значення параметру в алгоритмі Свена зображено на рис. 17.

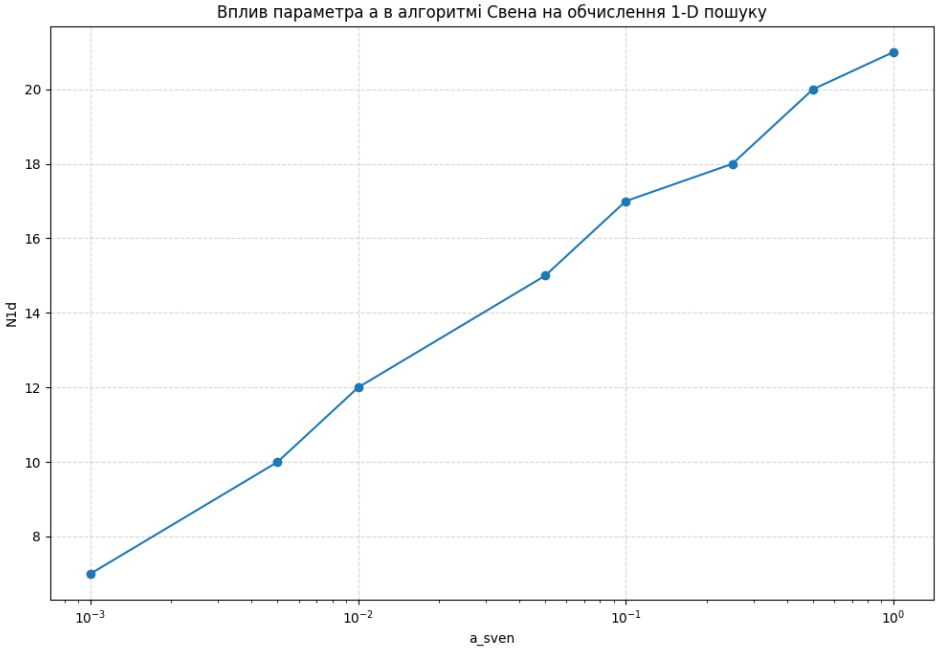


Рис. 17. Кількість обчислень цільової функції відповідно до значення параметра Свена

Як можна побачити з табл. 9, у якій зображено перевірку збіжності для лінійно-обмеженої області пошуку використовуючи різні параметри Свена, немає жодної різниці в кінцевому значенні локального мінімуму, що й очікувалося. Проте, кількість обчислень цільової функції відрізняється, збільшуючись, коли параметр дуже великий. Отже, найкраще використовувати значення параметра Свена від 0,001 до 0,01 для мінімізації функції Розенброка з заданими початковими умовами.

Таблиця 9. **Перевірка збіжності для лінійно-обмеженої області пошуку використовуючи різні параметри Свена**

α	n_{1d}	x_1, x_2	$f\left(x_1, x_2\right)$
0,001	7	$\begin{cases} x_1 = 0.8231812 \\ x_2 = 0.6768188 \end{cases}$	0.0313303
0,005	10	$\begin{cases} x_1 = 0.8231998 \\ x_2 = 0.6768002 \end{cases}$	0.0313319
0,01	12	$\begin{cases} x_1 = 0.8232290 \\ x_2 = 0.6767710 \end{cases}$	0.0313354
0,05	15	$\begin{cases} x_1 = 0.8232565 \\ x_2 = 0.6767435 \end{cases}$	0.0313398
0,1	17	$\begin{cases} x_1 = 0.8230747 \\ x_2 = 0.6769253 \end{cases}$	0.0313303
0,25	18	$\begin{cases} x_1 = 0.8232798 \\ x_2 = 0.6767202 \end{cases}$	0.0313444
0,5	20	$\begin{cases} x_1 = 0.8230693 \\ x_2 = 0.6769307 \end{cases}$	0.0313307
1	21	$\begin{cases} x_1 = 0.8232574 \\ x_2 = 0.6767426 \end{cases}$	0.0313400

Для порівняння покращення точності результатів, оновленого критерію закінчення, на рис. 18 та 19 представлені траєкторії пошуку мінімуму за допомогою методу спуску Розенброка та одновимірного методу ДСК-Пауелла і залежність методу ДСК-Пауелла від параметра в алгоритмі Свена.

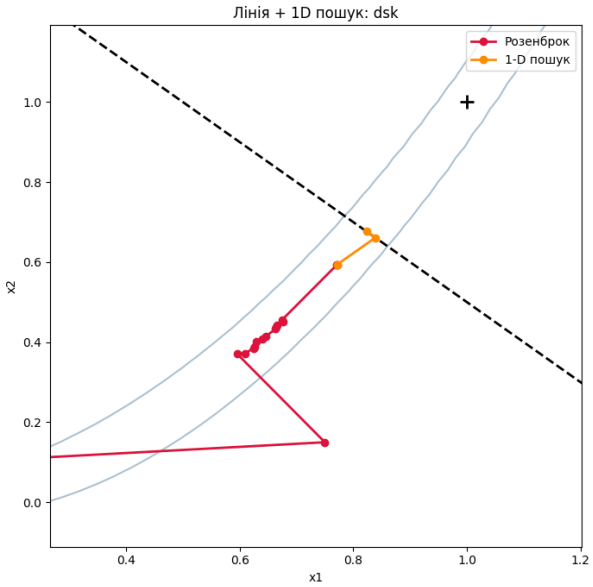


Рис. 18. Траєкторії пошуку локального мінімуму за допомогою методу ДСК-Пауелла

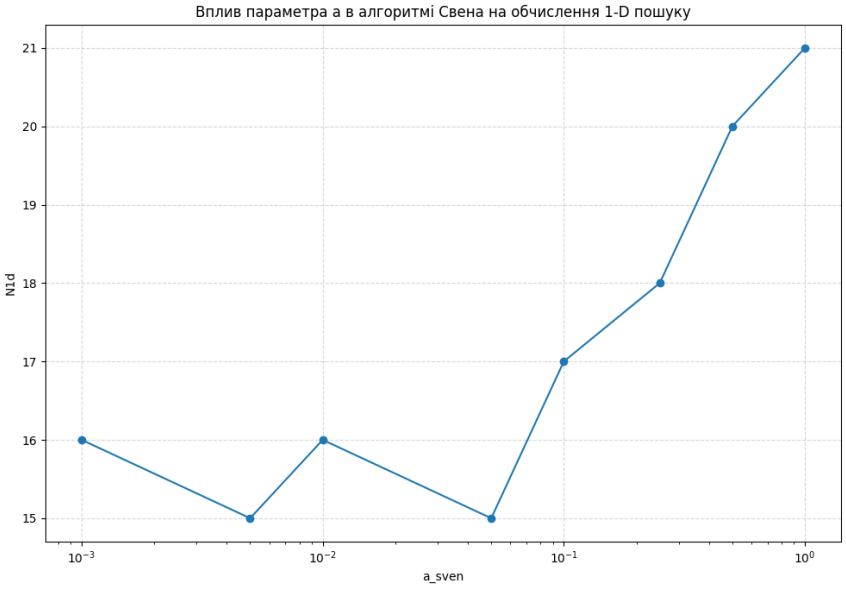


Рис. 19. Кількість обчислень цільової функції відповідно до значення параметра Свена

ВИСНОВКИ

Під час виконання дослідження для задачі безумовної оптимізації було встановлено такі значення параметрів для мінімізації функції Розенброка за допомогою методу Розенброка за 578 обчислень цільової функції з точністю $\varepsilon = 10^{-11}$: $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$ і отриманими значеннями $\begin{cases} x_1 = 1 \\ x_2 = 1 \end{cases}$, $f = 4{,}4 \cdot 10^{-16}$. Відповідно це – найкращий набір параметрів за табл. 3, при якому функція збігається за дуже малу кількість обчислень цільової функції для $\varepsilon = 10^{-11}$.

Було виявлено, що різниці між використанням критерію закінчення $\begin{cases} \frac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \left| f\left(x^{k+1}\right) - f\left(x^k\right) \right| \le \varepsilon \end{cases}$ або $\|\Delta x\| \le \varepsilon$ для мінімізації кількості обчислень цільової функції немає. Також, були досліджені модифікації методу, але вони не приносять жодного результату для комбінацій параметрів, які є вже є найкращими для своєї похибки.

У задачі умовної оптимізації метод Розенброка було використано як метод спуску з урахуванням допустимої області. Дослідження показало, що характер розв’язку істотно залежить від розташування безумовного мінімуму відносно допустимої області та від виду самих обмежень. Відповідно, для оновленого критерію закінчення для умовної оптимізації – $\begin{cases} \frac{\|x^{k+1} - x^k\|}{\|x^k\|} \le \varepsilon \\ \frac{\left| f\left(x^{k+1}\right) - f\left(x^k\right) \right|}{\left| f\left(x^k\right) \right|} \le \varepsilon \end{cases}$, було виявлено, що кількість обчислень цільової функції буде 300 – якщо мінімум поза опуклою областю з похибкою 0,0002374, 418 – якщо мінімум поза лінійно обмеженою областю з похибкою 0,0000081, 540 – якщо мінімум поза

неопуклою областю з похибкою 0,00045765 (точка знайденого мінімуму залежить від початкової точки та параметрів методу). Точність для дослідження умовної оптимізації – $\varepsilon = 10^{-3}$, параметри методу: $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$.

Було виявлено, що метод одновимірного пошуку ДСК-Пауелл використовував лише 8 ітерацій, коли метод Золотого перетину використовував 17 ітерацій для мінімізації функції з лінійними обмеженнями. Відповідні отримані значення функцій: 0,0313300 і 0,0313283. Також, було встановлено, що для одновимірних методів практично достатньою виявилася точність $\varepsilon = 10^{-4}$, при якій зберігається мала кількість ітерацій (8 і 21 відповідно) і обидва методи дають практично однакове значення функції 0,03133.

Було визначено, що найкраще використовувати значення параметра Свена від 0,001 до 0,01 для мінімізації функції Розенброка з заданими початковими умовами $\begin{pmatrix} \Delta x_1 = 1{,}95 \\ \Delta x_2 = 0{,}15 \end{pmatrix}$, $\begin{cases} \alpha = 4{,}5 \\ \beta = -0{,}25 \end{cases}$, оскільки кількість ітерацій одновимірного пошуку ДСК-Пауелла буде в інтервалі від 7 до 12 з відповідними значеннями функцій від 0,0313303 до 0,0313354. При $\alpha_{sv} > 0{,}01$ число ітерацій збільшувалося до 21 без помітного покращення значення функції.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

Ладогубець Т. С. Чисельні методи оптимізації : з дисципліни «Методи оптимізації» [Електронний ресурс] : практикум : навч. посібник за освітньою програмою «Наука про дані та математичне моделювання» спец. 113 Прикладна математика / Т. С. Ладогубець, О. Д. Фіногенов, А. М. Губський ; КПІ ім. Ігоря Сікорського. – 2-ге вид., переробл. та допов. – Електрон. текст. дані (1 файл: 3,2 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2024. – 125 с.

Химмельблау Д. Прикладное нелинейное программирование / Химмельблау Д. — М. : Мир, 1975. — 535 с.

ДОДАТКИ

Додаток А

Код програми безумовної оптимізації

```
import numpy as np
import matplotlib.pyplot as plt


def f(x1, x2):
    return 100 * (x1 ** 2 - x2) ** 2 + (x1 - 1) ** 2

def rosenbrock(x1, x2, l, a, b, n, n_rosenbrock, f_comp, fail1, fail2, success1,
success2, verbose, is_initial, modification, rec_depth=0, max_rec_depth=500):
    if modification:
        if rec_depth > 4:
            if a < 3:
                a *= 2
            else:
                a /= 2
            if b ** 2 * a > 0.5:
                b /= 2
            else:
                b *= 2

    if rec_depth >= max_rec_depth:
        x = [x1, x2]
        return x, l, np.nan
    if f(x1 + l[0] * s1[0], x2 + l[0] * s1[1]) < f_comp:
        n += 1
        x1 += l[0] * s1[0]
        x2 += l[0] * s1[1]
        success1 = True
        if verbose:
            print(f"{n:^3} | {l[0]:^11.3f} | {x1:^11.3f} | {x2:^11.3f} | {f(x1,
x2):^11.3f} | {f(x1, x2) < f_comp}")
        f_comp = f(x1, x2)
        if f(x1 + l[1] * s2[0], x2 + l[1] * s2[1]) < f_comp:
            n += 1
            x1 += l[1] * s2[0]
            x2 += l[1] * s2[1]
            success2 = True
            if verbose:
                print(f"{n:^3} | {l[1]:^11.3f} | {x1:^11.3f} | {x2:^11.3f} | {f(x1,
x2):^11.3f} | {f(x1, x2) < f_comp}")
            f_comp = f(x1, x2)
            l = [l[0] * a, l[1] * a]
            return rosenbrock(x1, x2, l, a, b, n, n_rosenbrock, f_comp, fail1, fail2,
success1, success2, verbose, is_initial, modification, rec_depth + 1, max_rec_depth)
        else:
            n += 1
            if success2 == True:
                fail2 = True
            if verbose:
                print(f"{n:^3} | {l[1]:^11.3f} | {x1 + l[1] * s2[0]:^11.3f} | {x2 + l[1]
* s2[1]:^11.3f} | {f(x1 + l[1] * s2[0], x2 + l[1] * s2[1]):^11.3f} | {f(x1 + l[1] *
s2[0], x2 + l[1] * s2[1]) < f_comp}")
            l = [l[0] * a, l[1] * b]
            return rosenbrock(x1, x2, l, a, b, n, n_rosenbrock, f_comp, fail1, fail2,
success1, success2, verbose, is_initial, modification, rec_depth + 1, max_rec_depth)
    else:
        n += 1
        if success1 == True:
            fail1 = True
```

Продовження додатку А

```
        if verbose:
            print(f"{n:^3} | {l[0]:^11.3f} | {x1 + l[0] * s1[0]:^11.3f} | {x2 + l[0] *
s1[1]:^11.3f} | {f(x1 + l[0] * s1[0], x2 + l[0] * s1[1]):^11.3f} | {f(x1 + l[0] * s1[0],
x2 + l[0] * s1[1]) < f_comp}")
        if f(x1 + l[1] * s2[0], x2 + l[1] * s2[1]) < f_comp:
            n += 1
            x1 += l[1] * s2[0]
            x2 += l[1] * s2[1]
            success2 = True
            if verbose:
                print(f"{n:^3} | {l[1]:^11.3f} | {x1:^11.3f} | {x2:^11.3f} | {f(x1,
x2):^11.3f} | {f(x1, x2) < f_comp}")
            f_comp = f(x1, x2)
            l = [l[0] * b, l[1] * a]
            return rosenbrock(x1, x2, l, a, b, n, n_rosenbrock, f_comp, fail1, fail2,
success1, success2, verbose, is_initial, modification, rec_depth + 1, max_rec_depth)
        else:
            n += 1
            if success2 == True:
                fail2 = True
            if verbose:
                print(f"{n:^3} | {l[1]:^11.3f} | {x1 + l[1] * s2[0]:^11.3f} | {x2 + l[1]
* s2[1]:^11.3f} | {f(x1 + l[1] * s2[0], x2 + l[1] * s2[1]):^11.3f} | {f(x1 + l[1] *
s2[0], x2 + l[1] * s2[1]) < f_comp}")
            l = [l[0] * b, l[1] * b]
            if fail1 == True and fail2 == True:
                if is_initial:
                    l = [l[0], l[1]]
                else:
                    l = [l[0] / a * b, l[1] / a * b]
                x = [x1, x2]
                n_rosenbrock += 1
                return x, l, n, n_rosenbrock
            return rosenbrock(x1, x2, l, a, b, n, n_rosenbrock, f_comp, fail1, fail2,
success1, success2, verbose, is_initial, modification, rec_depth + 1, max_rec_depth)

def params(x, x_old, s1, s2):
    a1 = [(x[0] - x_old[0]) * s1[0] + (x[1] - x_old[1]) * s2[0], (x[0] - x_old[0]) *
s1[1] + (x[1] - x_old[1]) * s2[1]]
    a2 = [(x[1] - x_old[1]) * s2[0], (x[1] - x_old[1]) * s2[1]]
    a1_norm = np.sqrt(a1[0] ** 2 + a1[1] ** 2)
    if a1_norm < 1e-12:
        return s1, s2
    s1 = [a1[0] / a1_norm, a1[1] / a1_norm]
    b2 = [a2[0] - (a2[0] * s1[0] + a2[1] * s1[1]) * s1[0],
          a2[1] - (a2[0] * s1[0] + a2[1] * s1[1]) * s1[1]]
    b2_norm = np.sqrt(b2[0] ** 2 + b2[1] ** 2)
    if b2_norm < 1e-12:
        s2 = [-s1[1], s1[0]]
    else:
        s2 = [b2[0] / b2_norm, b2[1] / b2_norm]
    return s1, s2

def output(x, l, n, n_rosenbrock, a, b, s1, s2, x_old, modification, verbose):
    if n == 0:
        if verbose:
            print(f"f({x[0]:.3f}, {x[1]:.3f}) = {f(x[0], x[1]):.3f}")

    x_old = x

    if verbose:
        print("----+-------------+-------------+-------------+-------------
+-----------")
        print("n   |      l      |      x1     |      x2     |  f(x1, x2)  |  Success")
        print("----+-------------+-------------+-------------+-------------
+-----------")
    if n == 0:
        x, l, n, n_rosenbrock  = rosenbrock(x[0], x[1], l, a, b, n, n_rosenbrock,
f(x[0], x[1]), False, False, False, False, verbose, True, modification)
    else:
```

Продовження додатку А

```
        x, l, n, n_rosenbrock  = rosenbrock(x[0], x[1], l, a, b, n, n_rosenbrock,
f(x[0], x[1]), False, False, False, False, verbose, False, modification)
    if verbose:
        print("----+------------+------------+------------+------------+----------
\n")
        print(f"f({x[0]:4f}, {x[1]:4f}) = {f(x[0], x[1]):.2e}, l = [{l[0]:.3e},
{l[1]:.3e}]")
    s1, s2 = params(x, x_old, s1, s2)
    return x, l, n, n_rosenbrock, a, b, s1, s2, x_old

def stop1(e):
    step_norm = np.sqrt((x[0] - x_old[0]) ** 2 + (x[1] - x_old[1]) ** 2)
    x_norm = np.sqrt(x[0] ** 2 + x[1] ** 2)
    if x_norm < 1e-12:
        x_norm = 1.0
    if step_norm < 1e-12:
        return False
    if abs(step_norm) / x_norm < e and abs(f(x[0], x[1]) - f(x_old[0], x_old[1])) < e:
            return True
    return False

def stop2(eps):
    if np.sqrt((x[0] - x_old[0]) ** 2 + (x[1] - x_old[1]) ** 2) < eps:
        return True
    return False

def stalled_state():
    step_norm = np.sqrt((x[0] - x_old[0]) ** 2 + (x[1] - x_old[1]) ** 2)
    return step_norm < 1e-12

def run_case(dx1, dx2, a_value, b_value, eps_value, modification=True):
    global x, x_old, s1, s2, a, b, e

    x = [-1.2, 0]
    x_old = [-1.2, 0]
    s1 = [1, 0]
    s2 = [0, 1]
    n = 0
    l = [dx1, dx2]
    a = a_value
    b = b_value
    e = eps_value

    while True:
        x, l, n, a, b, s1, s2, x_old = output(x, l, n, a, b, s1, s2, x_old,
modification, False)
        if isinstance(n, float) and np.isnan(n):
            return np.nan, [np.nan, np.nan], np.nan
        if stop1(eps_value):
            return n, x.copy(), f(x[0], x[1])
        if stalled_state():
            return np.nan, [np.nan, np.nan], np.nan

def refine_values(center, left_factor=0.5, right_factor=1.5, points=5,
min_positive=1e-3):
    vals = np.linspace(center * left_factor, center * right_factor, points)
    vals = [round(max(v, min_positive), 3) for v in vals]
    return sorted(set(vals))

if __name__ == "__main__":
    l = [1.95, 0.15]
    a = 4.5
    b = -0.25
    e = 1e-3

    s1 = [1, 0]
    s2 = [0, 1]

    x = [-1.2, 0]
    x_old = x.copy()
```

Продовження додатку А

```
    x_1 = [x[0]]
    x_2 = [x[1]]

    n = 0
    n_rosenbrock = 0
    while True:
        x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n, n_rosenbrock, a, b,
s1, s2, x_old, False, False)
        x_1.append(x[0])
        x_2.append(x[1])
        if isinstance(n, float) and np.isnan(n):
            break
        if stop1(e) == True:
            break
        if stalled_state():
            break

    print(f"n = {n}, n_rosenbrock = {n_rosenbrock}")
    grid = np.linspace(-2, 2, 100)
    X, Y = np.meshgrid(grid, grid)
    Z = f(X, Y)


    plt.figure(figsize=(7, 6))
    plt.contour(X, Y, np.log10(Z + 1e-10), levels=2, cmap='viridis', alpha=0.6)

    plt.plot(x_1, x_2, label="x", marker="o", markersize=6, linewidth=2, color="red")
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.title(f"F(x) = {f(x[0], x[1]):.2e}, N = {n}")
    plt.grid(alpha=0.4, linestyle='--', color='gray', linewidth=0.5)
    plt.show()

    labs_visualize = False
    eps_visualize = False
    global_search = False

    minimum = None

    if labs_visualize:
        dx_test = [0.01, 0.05, 0.1, 0.15, 0.25, 0.5, 0.75, 1, 1.5, 2, 2.5, 3]

        n_arr = []

        for i in dx_test:
            for j in dx_test:
                x = [-1.2, 0]
                x_old = [-1.2, 0]
                s1 = [1, 0]
                s2 = [0, 1]
                n = 0
                l = [i, j]
                while True:
                    x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n,
n_rosenbrock, a, b, s1, s2, x_old, False, False)
                    if isinstance(n, float) and np.isnan(n):
                        n_arr.append(np.nan)
                        break
                    if stop1(e) == True:
                        n_arr.append(n)
                        break
                    if stalled_state():
                        n_arr.append(np.nan)
                        break

        x_axis = dx_test
        y_axis = dx_test[::-1]

        values = np.zeros((len(dx_test), len(dx_test)), dtype=float)
```

Продовження додатку А

```
        for i in range(len(n_arr)):
            values[i // len(dx_test)][i % len(dx_test)] = float(n_arr[i])

        values_plot = values[::-1]
        fig, ax = plt.subplots(figsize=(14, 8))
        im = ax.imshow(values_plot, cmap='viridis', interpolation='nearest')
        cbar = plt.colorbar(im, label="number of iterations")

        ax.set_xticks(np.arange(len(dx_test)), labels=dx_test)
        ax.set_yticks(np.arange(len(dx_test)), labels=dx_test[::-1])

        plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
         rotation_mode="anchor")

        for i in range(len(dx_test)):
            for j in range(len(dx_test)):
                cell_value = values_plot[i, j]
                label = "NaN" if np.isnan(cell_value) else int(cell_value)
                text = ax.text(j, i, label,
                       ha="center", va="center", color="white")

        plt.xlabel("dx2")
        plt.ylabel("dx1")

        fig.tight_layout()
        plt.show()

        minimum = [np.inf, 0, 0, 0, 0]
        for i in range(len(dx_test)):
            for j in range(len(dx_test)):
                if not np.isnan(n_arr[i * len(dx_test) + j]):
                    if n_arr[i * len(dx_test) + j] < minimum[0]:
                        minimum = [n_arr[i * len(dx_test) + j], dx_test[i], dx_test[j],
a, b]


        a_test = [1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 7, 10, 12, 15]
        b_test = [-0.1, -0.15, -0.2, -0.25, -0.3, -0.35, -0.4, -0.5, -0.75, -0.9]

        dx_test_i = [2, 2, 0.5, 0.25, 0.15, 0.15]
        dx_test_j = [0.1, 0.25, 0.5, 0.25, 0.1, 0.5]

        for k in range(len(dx_test_i)):
         n_arr = []
         for i in a_test:
            for j in b_test:
                x = [-1.2, 0]
                x_old = [-1.2, 0]
                s1 = [1, 0]
                s2 = [0, 1]
                n = 0
                l = [dx_test_i[k], dx_test_j[k]]
                a = i
                b = j
                while True:
                    x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n,
n_rosenbrock, a, b, s1, s2, x_old, False, False)
                    if isinstance(n, float) and np.isnan(n):
                        n_arr.append(np.nan)
                        break
                    if stop1(e) == True:
                        n_arr.append(n)
                        break
                    if stalled_state():
                        n_arr.append(np.nan)
                        break

         x_axis = a_test
         y_axis = b_test[::-1]
```

Продовження додатку А

```
            values = np.zeros((len(b_test), len(a_test)), dtype=float)

            idx = 0
            for i in range(len(a_test)):
                for j in range(len(b_test)):
                    values[j, i] = float(n_arr[idx])
                    idx += 1

            values_plot = values[::-1]
            fig, ax = plt.subplots(figsize=(14, 8))
            im = ax.imshow(values_plot, cmap='viridis', interpolation='nearest')

            ax.set_xticks(np.arange(len(a_test)), labels=a_test)
            ax.set_yticks(np.arange(len(b_test)), labels=b_test[::-1])

            plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
                rotation_mode="anchor")

            for i in range(len(b_test)):
                for j in range(len(a_test)):
                    cell_value = values_plot[i, j]
                    label = "NaN" if np.isnan(cell_value) else int(cell_value)
                    text = ax.text(j, i, label,
                           ha="center", va="center", color="white")

            plt.xlabel("a")
            plt.ylabel("b")
            cbar = plt.colorbar(im, label="number of iterations")
            fig.tight_layout()
            plt.show()

            for i in range(len(a_test)):
                for j in range(len(b_test)):
                    if not np.isnan(n_arr[i * len(b_test) + j]):
                        if n_arr[i * len(b_test) + j] < minimum[0]:
                            minimum = [n_arr[i * len(b_test) + j], minimum[1], minimum[2],
a_test[i], b_test[j]]

        stops = [1, 2]
        n_arr = []
        labels = []

        for i in stops:
            l_l = [[2, 0.1], [0.15, 0.5]]
            a_l = [10, 7]
            b_l = [-0.25, -0.4]
            for j in range(2):
                x = [-1.2, 0]
                x_old = [-1.2, 0]
                s1 = [1, 0]
                s2 = [0, 1]
                n = 0
                l = l_l[j]
                a = a_l[j]
                b = b_l[j]
                while True:
                    x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n,
n_rosenbrock, a, b, s1, s2, x_old, False, False)
                    if i == 1:
                        if stop1(e) == True:
                            n_arr.append(n)
                            break
                        if stalled_state():
                            n_arr.append(np.nan)
                            break
                    if i == 2:
                        if stop2(e) == True:
                            n_arr.append(n)
                            break
```

Продовження додатку А

```
                    if stalled_state():
                        n_arr.append(np.nan)
                        break
                labels.append(f"Stop {i}\nParam {j+1}")

        plt.figure(figsize=(10, 6))
        x_positions = range(len(n_arr))
        plt.bar(x_positions, n_arr, color=['skyblue', 'steelblue', 'salmon', 'tomato'])

        plt.xlabel("Configuration")
        plt.ylabel("n")
        plt.xticks(x_positions, labels) # Map the 4 labels to the 4 bars

        # Add the actual values on top of the bars for clarity
        for idx, val in enumerate(n_arr):
            if not np.isnan(val):
                plt.text(idx, val, f'{int(val)}', ha='center', va='bottom')

        plt.tight_layout()
        plt.show()


    if eps_visualize:
        e = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9, 1e-10, 1e-11]
        n_arr = []
        x_1_arr = []
        x_2_arr = []
        n_rosenbrock_arr = []

        n_arr2 = []
        x_1_arr2 = []
        x_2_arr2 = []
        n_rosenbrock_arr2 = []

        for i in e:
            x = [-1.2, 0]
            x_old = [-1.2, 0]
            n_rosenbrock = 0
            s1 = [1, 0]
            s2 = [0, 1]

            l = [1.95, 0.15]
            a = 4.5
            b = -0.25

            n = 0
            eps = i
            while True:
                x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n,
n_rosenbrock, a, b, s1, s2, x_old, False, False)
                if stop1(eps) == True:
                    n_arr.append(n)
                    n_rosenbrock_arr.append(n_rosenbrock)
                    x_1_arr.append(x[0])
                    x_2_arr.append(x[1])
                    break
                if stalled_state():
                    n_arr.append(np.nan)
                    n_rosenbrock_arr.append(np.nan)
                    x_1_arr.append(np.nan)
                    x_2_arr.append(np.nan)
                    break

        for i in e:
            x = [-1.2, 0]
            x_old = [-1.2, 0]
            n_rosenbrock = 0
            s1 = [1, 0]
            s2 = [0, 1]
```

Продовження додатку А

```
            l = [1.95, 0.15]
            a = 4.5
            b = -0.25

            n = 0
            eps = i
            while True:
                x, l, n, n_rosenbrock, a, b, s1, s2, x_old = output(x, l, n,
n_rosenbrock, a, b, s1, s2, x_old, False, False)
                if stop2(eps) == True:
                    n_arr2.append(n)
                    n_rosenbrock_arr2.append(n_rosenbrock)
                    x_1_arr2.append(x[0])
                    x_2_arr2.append(x[1])
                    break
                if stalled_state():
                    n_arr2.append(np.nan)
                    n_rosenbrock_arr2.append(np.nan)
                    x_1_arr2.append(np.nan)
                    x_2_arr2.append(np.nan)
                    break

        x_crit = 0
        for i in n_arr:
            if i > 600:
                x_crit = n_arr.index(i)
                break
        x_crit2 = 0
        for i in n_arr2:
            if i > 600:
                x_crit2 = n_arr2.index(i)
                break
        if x_crit2 > x_crit:
            x_crit = x_crit2

        plt.figure(figsize=(8, 5))
        plt.plot(e[::-1], n_arr, marker="o")
        plt.plot(e[::-1], n_arr2, marker="o")
        plt.xscale("log")
        plt.xlabel("eps")
        plt.ylabel("n")
        plt.legend(["Кр. зуп. 1", "Кр. зуп. 2"], loc="lower right")
        plt.xticks(e, [f"1e-{13-i}" for i in range(2, 13)])
        plt.hlines(600, 0, 1e-1, linestyle="--", color="red")
        if x_crit > 0:
            plt.vlines(0.1 ** (12 - x_crit - 1), 0, max(n_arr), linestyle="--",
color="red")
        plt.grid(True, which="major", linestyle="--", alpha=0.5)
        plt.tight_layout()
        plt.show()
        print(n_rosenbrock_arr)

        print("-------+-----------+-------------+-------+-------+")
        print("   e   |     x     |      f      |  n_r  |   n   |")
        print("-------+-----------+-------------+-------+-------+")
        for i in range(len(e)):
            print(f"{e[i]:^6.0e} | {x_1_arr[i]:^9.7f} | {f(x_1_arr[i],
x_2_arr[i]):^11.3e} | {n_arr[i]:^5.0f} | {n_rosenbrock_arr[i]:^5.0f} |")
            print(f"       | {x_2_arr[i]:^9.7f} |             |       |       |")
            print("-------+-----------+-------------+-------+-------+")
            print(f"{e[i]:^6.0e} | {x_1_arr2[i]:^9.7f} | {f(x_1_arr2[i],
x_2_arr2[i]):^11.3e} | {n_arr2[i]:^5.0f} | {n_rosenbrock_arr2[i]:^5.0f} |")
            print(f"       | {x_2_arr[i]:^9.7f} |             |       |       |")
            print("-------+-----------+-------------+-------+-------+")


    if global_search:
        e_test = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9, 1e-10]
        dx_test_1 = np.arange(0.15, 2, 0.1)
```

Продовження додатку А

```
        dx_test_2 = np.arange(0.05, 0.5, 0.05)
        a_test = np.arange(3, 11, 0.5)
        b_test = np.arange(-0.4, -0.1, 0.05)

        global_results = []

        for eps_value in e_test:
            global_minimum = [np.inf, np.nan, np.nan, np.nan, np.nan, [np.nan, np.nan],
np.nan]

            for dx1 in dx_test_1:
                for dx2 in dx_test_2:
                    for a_try in a_test:
                        for b_try in b_test:
                            n_value, x_value, f_value = run_case(dx1, dx2, a_try, b_try,
eps_value, False)

                            if not (isinstance(n_value, float) and np.isnan(n_value)):
                                if n_value < global_minimum[0]:
                                    global_minimum = [n_value, dx1, dx2, a_try, b_try,
x_value, f_value]
                                    print(f"New global minimum for eps = {eps_value}: n
= {n_value}")

            global_results.append([eps_value] + global_minimum)

        print("--------+-------+---------+---------+-------+--------+-------------
+-------------+-------------")
        print("   eps  |   n   |   dx1   |   dx2   |   a   |   b    |     x1      |
x2       |    f(x)     ")
        print("--------+-------+---------+---------+-------+--------+-------------
+-------------+-------------")
        for eps_value, n_value, dx1, dx2, a_value, b_value, x_value, f_value in
global_results:
            if isinstance(n_value, float) and np.isnan(n_value):
                print(f"{eps_value:^8.0e}| {'NaN':^7} | {'NaN':^7} | {'NaN':^7} |
{'NaN':^5} | {'NaN':^6} | {'NaN':^11} | {'NaN':^11} | {'NaN':^11}")
            else:
                print(
                    f"{eps_value:^8.0e}| {int(n_value):^7} | {dx1:^7.3f} | {dx2:^7.3f} |
"
                    f"{a_value:^5.2f} | {b_value:^6.2f} | {x_value[0]:^11.7f} |
{x_value[1]:^11.7f} | {f_value:^11.9f}"
                )
        print("--------+-------+---------+---------+-------+--------+-------------
+-------------+-------------")
```

Додаток Б

Код програми умовної оптимізації

```
import numpy as np
import matplotlib.pyplot as plt


OBJECTIVE_EVALS = 0


def fmt_float(value, digits=7):
    return f"{float(value):.{digits}f}"


def rosenbrock_function(x):
    global OBJECTIVE_EVALS
    OBJECTIVE_EVALS += 1
    x1, x2 = np.array(x, dtype=float)
    return 100 * (x1 ** 2 - x2) ** 2 + (x1 - 1) ** 2


def circular_constraint(radius, center=(0.0, 0.0)):
    def g(x):
        return (x[0] - center[0]) ** 2 + (x[1] - center[1]) ** 2 - radius ** 2

    g.kind = "circle"
    g.radius = radius
    g.center = np.array(center, dtype=float)
    return g


def linear_constraint(a1, a2, b):
    def g(x):
        return a1 * x[0] + a2 * x[1] - b

    g.kind = "line"
    g.a1 = a1
    g.a2 = a2
    g.b = b
    return g


def nonconvex_constraint():
    def g(x):
        outer_circle = x[0] ** 2 + x[1] ** 2 - 4.0
        inner_hole = 0.10 - ((x[0] - 1.0) ** 2 + (x[1] - 1.0) ** 2)
        return max(outer_circle, inner_hole)

    g.kind = "nonconvex"
    return g


def is_feasible(x, constraints, tol=1e-12):
    point = np.array(x, dtype=float)
    return all(g(point) <= tol for g in constraints)


def violation_value(x, constraints):
    point = np.array(x, dtype=float)
    if not constraints:
        return 0.0
    return max(max(0.0, g(point)) for g in constraints)


def project_to_feasible(x, constraints):
    x = np.array(x, dtype=float)
    for g in constraints:
        if getattr(g, "kind", "") == "circle":
            diff = x - g.center
```

Продовження додатку Б

```
            norm_diff = np.linalg.norm(diff)
            if norm_diff > g.radius:
                x = g.center + diff * (g.radius / max(norm_diff, 1e-12))
        elif getattr(g, "kind", "") == "line":
            normal = np.array([g.a1, g.a2], dtype=float)
            value = np.dot(normal, x) - g.b
            if value > 0:
                x = x - (value / max(np.dot(normal, normal), 1e-12)) * normal
        elif getattr(g, "kind", "") == "nonconvex":
            norm_x = np.linalg.norm(x)
            if norm_x > 2.0:
                x = x * (2.0 / max(norm_x, 1e-12))
            center = np.array([1.0, 1.0], dtype=float)
            hole_radius = np.sqrt(0.10)
            diff = x - center
            diff_norm = np.linalg.norm(diff)
            if diff_norm < hole_radius:
                if diff_norm < 1e-12:
                    diff = np.array([1.0, 0.0], dtype=float)
                    diff_norm = 1.0
                x = center + diff * (hole_radius / diff_norm)
    return x


def update_basis(step1, step2, s1, s2):
    a1 = np.array(step1, dtype=float) + np.array(step2, dtype=float)
    a2 = np.array(step2, dtype=float)
    a1_norm = np.linalg.norm(a1)
    if a1_norm < 1e-12:
        return np.array(s1, dtype=float), np.array(s2, dtype=float)

    new_s1 = a1 / a1_norm
    b2 = a2 - np.dot(a2, new_s1) * new_s1
    b2_norm = np.linalg.norm(b2)
    if b2_norm < 1e-12:
        new_s2 = np.array([-new_s1[1], new_s1[0]], dtype=float)
    else:
        new_s2 = b2 / b2_norm
    return new_s1, new_s2


def one_dimensional_function(func, x, direction):
    x = np.array(x, dtype=float)
    direction = np.array(direction, dtype=float)
    return lambda lam: func(x + lam * direction)


def sven_search(phi, x0=0.0, d=0.1, max_iter=60):
    y0 = phi(x0)
    x_left = x0 - d
    x_right = x0 + d
    y_left = phi(x_left)
    y_right = phi(x_right)

    if y_left >= y0 and y_right >= y0:
        return min(x_left, x_right), max(x_left, x_right)

    if y_left < y0:
        sign = -1.0
        points = [x0, x_left]
        values = [y0, y_left]
    else:
        sign = 1.0
        points = [x0, x_right]
        values = [y0, y_right]

    count = 1
    while count <= max_iter:
        step = d * 2 ** count
```

Продовження додатку Б

```
            x_new = points[-1] + sign * step
            y_new = phi(x_new)
            if y_new < values[-1]:
                points.append(x_new)
                values.append(y_new)
                count += 1
                continue

            x_prev = points[-1]
            x_mid = (x_prev + x_new) / 2.0
            return min(points[-2], x_new, x_mid), max(points[-2], x_new, x_mid)

        return min(points), max(points)


    def golden_section(phi, a, b, eps=1e-6, max_iter=200):
        x1 = a + 0.382 * (b - a)
        x2 = a + 0.618 * (b - a)
        y1 = phi(x1)
        y2 = phi(x2)

        while (b - a) > eps and max_iter > 0:
            if y1 < y2:
                b = x2
                x2 = x1
                y2 = y1
                x1 = a + 0.382 * (b - a)
                y1 = phi(x1)
            else:
                a = x1
                x1 = x2
                y1 = y2
                x2 = a + 0.618 * (b - a)
                y2 = phi(x2)
            max_iter -= 1

        return (a + b) / 2.0


    def dsk_powell(phi, a, b, eps=1e-6, max_iter=100):
        triple = [a, (a + b) / 2.0, b]
        values = [phi(triple[0]), phi(triple[1]), phi(triple[2])]

        for _ in range(max_iter):
            x1, x2, x3 = triple
            f1, f2, f3 = values

            denom_12 = x2 - x1
            denom_13 = x3 - x1
            denom_23 = x3 - x2
            if abs(denom_12) < 1e-12 or abs(denom_13) < 1e-12 or abs(denom_23) < 1e-12:
                return x2

            a1 = (f2 - f1) / denom_12
            a2 = ((f3 - f1) / denom_13 - (f2 - f1) / denom_12) / denom_23

            if abs(a2) < 1e-12:
                xs = (x1 + x3) / 2.0
            else:
                xs = (x1 + x2) / 2.0 - a1 / (2.0 * a2)
                if xs < x1 or xs > x3:
                    xs = (x1 + x3) / 2.0

            ys = phi(xs)
            if abs(x2 - xs) <= eps and abs(f2 - ys) <= eps:
                return xs

            points = sorted([(x1, f1), (x2, f2), (x3, f3), (xs, ys)], key=lambda item:
    item[0])
```

Продовження додатку Б

```
        best_index = min(range(len(points)), key=lambda i: points[i][1])

        if best_index == 0:
            new_points = points[:3]
        elif best_index == len(points) - 1:
            new_points = points[-3:]
        else:
            new_points = points[best_index - 1:best_index + 2]

        triple = [item[0] for item in new_points]
        values = [item[1] for item in new_points]

    return triple[1]


def line_search_on_line(func, x, tangent, step0, method="golden", eps_1d=1e-6,
sven_alpha=1.0):
    phi = one_dimensional_function(func, x, tangent)
    delta = max(abs(step0), 1e-3, sven_alpha)
    a, b = sven_search(phi, x0=0.0, d=delta)

    if method == "golden":
        lam = golden_section(phi, a, b, eps_1d)
    elif method == "dsk":
        lam = dsk_powell(phi, a, b, eps_1d)
    else:
        raise ValueError("method must be 'golden' or 'dsk'")

    return lam, np.array(x, dtype=float) + lam * np.array(tangent, dtype=float)


def constrained_value(func, point, constraints):
    point = np.array(point, dtype=float)
    if not is_feasible(point, constraints):
        return np.inf
    return func(point)


def rosenbrock_stage_kr(
    func,
    constraints,
    x,
    lmbd,
    a,
    b,
    n,
    n_rosen,
    f_comp,
    fail1,
    fail2,
    success1,
    success2,
    s1,
    s2,
    is_initial,
    rec_depth=0,
    max_rec_depth=500,
):
    if rec_depth >= max_rec_depth:
        return np.array(x, dtype=float), np.array(lmbd, dtype=float), np.nan, n_rosen

    candidate1 = np.array(x, dtype=float) + lmbd[0] * s1
    f1 = constrained_value(func, candidate1, constraints)
    if f1 < f_comp:
        n += 1
        x = candidate1
        success1 = True
        f_comp = f1
```

Продовження додатку Б

```
            candidate2 = np.array(x, dtype=float) + lmbd[1] * s2
            f2 = constrained_value(func, candidate2, constraints)
            if f2 < f_comp:
                n += 1
                x = candidate2
                success2 = True
                f_comp = f2
                lmbd = np.array([lmbd[0] * a, lmbd[1] * a], dtype=float)
                return rosenbrock_stage_kr(
                    func, constraints, x, lmbd, a, b, n, n_rosen, f_comp,
                    fail1, fail2, success1, success2, s1, s2, is_initial,
                    rec_depth + 1, max_rec_depth
                )

            n += 1
            if success2:
                fail2 = True
            lmbd = np.array([lmbd[0] * a, lmbd[1] * b], dtype=float)
            return rosenbrock_stage_kr(
                func, constraints, x, lmbd, a, b, n, n_rosen, f_comp,
                fail1, fail2, success1, success2, s1, s2, is_initial,
                rec_depth + 1, max_rec_depth
            )

        n += 1
        if success1:
            fail1 = True

        candidate2 = np.array(x, dtype=float) + lmbd[1] * s2
        f2 = constrained_value(func, candidate2, constraints)
        if f2 < f_comp:
            n += 1
            x = candidate2
            success2 = True
            f_comp = f2
            lmbd = np.array([lmbd[0] * b, lmbd[1] * a], dtype=float)
            return rosenbrock_stage_kr(
                func, constraints, x, lmbd, a, b, n, n_rosen, f_comp,
                fail1, fail2, success1, success2, s1, s2, is_initial,
                rec_depth + 1, max_rec_depth
            )

        n += 1
        if success2:
            fail2 = True
        lmbd = np.array([lmbd[0] * b, lmbd[1] * b], dtype=float)
        if fail1 and fail2:
            if not is_initial:
                lmbd = np.array([lmbd[0] / a * b, lmbd[1] / a * b], dtype=float)
            n_rosen += 1
            return np.array(x, dtype=float), lmbd, n, n_rosen

        return rosenbrock_stage_kr(
            func, constraints, x, lmbd, a, b, n, n_rosen, f_comp,
            fail1, fail2, success1, success2, s1, s2, is_initial,
            rec_depth + 1, max_rec_depth
        )


    def params_kr_style(x, x_old, s1, s2):
        x = np.array(x, dtype=float)
        x_old = np.array(x_old, dtype=float)
        s1 = np.array(s1, dtype=float)
        s2 = np.array(s2, dtype=float)

        a1 = np.array([
            (x[0] - x_old[0]) * s1[0] + (x[1] - x_old[1]) * s2[0],
            (x[0] - x_old[0]) * s1[1] + (x[1] - x_old[1]) * s2[1],
```

Продовження додатку Б

```
    ], dtype=float)
    a2 = np.array([
        (x[1] - x_old[1]) * s2[0],
        (x[1] - x_old[1]) * s2[1],
    ], dtype=float)

    a1_norm = np.linalg.norm(a1)
    if a1_norm < 1e-12:
        return s1, s2

    new_s1 = a1 / a1_norm
    b2 = a2 - np.dot(a2, new_s1) * new_s1
    b2_norm = np.linalg.norm(b2)
    if b2_norm < 1e-12:
        new_s2 = np.array([-new_s1[1], new_s1[0]], dtype=float)
    else:
        new_s2 = b2 / b2_norm
    return new_s1, new_s2


def stop1_kr_style(x, x_old, func, eps):
    x = np.array(x, dtype=float)
    x_old = np.array(x_old, dtype=float)
    step_norm = np.linalg.norm(x - x_old)
    x_norm = np.linalg.norm(x)
    if x_norm < 1e-12:
        x_norm = 1.0
    if step_norm < 1e-12:
        return False
    return step_norm / x_norm < eps and abs(func(x) - func(x_old)) < eps


def stalled_state_kr_style(x, x_old):
    return np.linalg.norm(np.array(x, dtype=float) - np.array(x_old, dtype=float)) <
1e-12


def rosenbrock_direct(
    func,
    constraints,
    x0,
    dx0,
    a=3.0,
    b=-0.5,
    eps=1e-4,
    max_stages=500,
):
    x = project_to_feasible(x0, constraints)
    x_old = np.array(x, dtype=float).copy()
    lmbd = np.array(dx0, dtype=float)
    s1 = np.array([1.0, 0.0], dtype=float)
    s2 = np.array([0.0, 1.0], dtype=float)
    n = 0
    n_rosen = 0
    path = [x.copy()]

    for _ in range(max_stages):
        stage_start = np.array(x, dtype=float).copy()
        if n == 0:
            x, lmbd, n, n_rosen = rosenbrock_stage_kr(
                func, constraints, x, lmbd, a, b, n, n_rosen,
                constrained_value(func, x, constraints),
                False, False, False, False, s1, s2, True
            )
        else:
            x, lmbd, n, n_rosen = rosenbrock_stage_kr(
                func, constraints, x, lmbd, a, b, n, n_rosen,
                constrained_value(func, x, constraints),
                False, False, False, False, s1, s2, False
```

Продовження додатку Б

```
            )

        if isinstance(n, float) and np.isnan(n):
            return np.array(stage_start, dtype=float), lmbd, np.nan, path, True

        path.append(np.array(x, dtype=float).copy())
        prev_old = np.array(x_old, dtype=float).copy()
        x_old = np.array(stage_start, dtype=float).copy()
        s1, s2 = params_kr_style(x, x_old, s1, s2)

        if stop1_kr_style(x, x_old, func, eps):
            return np.array(x, dtype=float), lmbd, n, path, True

        if stalled_state_kr_style(x, x_old):
            return np.array(x, dtype=float), lmbd, n, path, True

        if np.allclose(prev_old, x_old) and np.allclose(x, stage_start):
            return np.array(x, dtype=float), lmbd, n, path, True

    return np.array(x, dtype=float), lmbd, n, path, False


def refine_on_linear_boundary(
    func,
    line_constraint_obj,
    x,
    dx,
    method="golden",
    eps_1d=1e-6,
    sven_alpha=0.1,
):
    global OBJECTIVE_EVALS
    normal = np.array([line_constraint_obj.a1, line_constraint_obj.a2], dtype=float)
    tangent = np.array([-line_constraint_obj.a2, line_constraint_obj.a1], dtype=float)
    tangent /= max(np.linalg.norm(tangent), 1e-12)

    value = np.dot(normal, x) - line_constraint_obj.b
    x_on_line = np.array(x, dtype=float) - (value / max(np.dot(normal, normal), 1e-12))
* normal
    step0 = max(np.linalg.norm(dx), 1e-3)
    evals_before = OBJECTIVE_EVALS
    _, x_best = line_search_on_line(func, x_on_line, tangent, step0, method, eps_1d,
sven_alpha)
    evals_used = OBJECTIVE_EVALS - evals_before
    return x_best, [x.copy(), x_on_line.copy(), x_best.copy()], evals_used


def coord_changes_from_paths(paths):
    count = 0
    for path in paths:
        for i in range(len(path) - 1):
            if not np.allclose(path[i], path[i + 1]):
                count += 1
    return count


def run_case(
    title,
    constraints,
    x0,
    dx0,
    a=3.0,
    b=-0.5,
    eps=1e-4,
    line_method=None,
    eps_1d=1e-6,
    sven_alpha=0.1,
):
    global OBJECTIVE_EVALS
```

Продовження додатку Б

```
    OBJECTIVE_EVALS = 0

    x, dx, n_evals, path, terminated_by_stall = rosenbrock_direct(
        rosenbrock_function,
        constraints,
        x0,
        dx0,
        a=a,
        b=b,
        eps=eps,
    )

    paths = [path]
    line_used = False
    one_d_evals = 0
    linear_constraints = [g for g in constraints if getattr(g, "kind", "") == "line"]
    if line_method is not None and linear_constraints and terminated_by_stall:
        x, line_path, one_d_evals = refine_on_linear_boundary(
            rosenbrock_function,
            linear_constraints[0],
            x,
            dx,
            method=line_method,
            eps_1d=eps_1d,
            sven_alpha=sven_alpha,
        )
        paths.append(line_path)
        line_used = True

    return {
        "title": title,
        "x": x,
        "f": rosenbrock_function(x),
        "violation": violation_value(x, constraints),
        "f_evals_rosen": int(n_evals),
        "f_evals_1d": int(one_d_evals),
        "f_evals_total": int(n_evals + one_d_evals),
        "coord_changes": coord_changes_from_paths(paths),
        "n_rosen": int(n_evals),
        "paths": paths,
        "line_used": line_used,
        "line_method": line_method,
        "terminated_by_stall": terminated_by_stall,
    }


def print_case_table(header, results):
    print()
    print(header)
    has_1d = any(result["f_evals_1d"] > 0 for result in results)
    if has_1d:
        print("----+------------------------------+--------+--------+--------
+--------+------------+------------+------------")
        print(" №  | Сценарій                     | Nrosen  |  N1d    | Nsum    | Ncoord
|     x1      |     x2      |    f(x)     ")
        print("----+------------------------------+--------+--------+--------
+--------+------------+------------+------------")
    else:
        print("----+------------------------------+--------+--------+------------
+------------+------------")
        print(" №  | Сценарій                     | Nf(x)   | Ncoord  |     x1      |
x2      |    f(x)     ")
        print("----+------------------------------+--------+--------+------------
+------------+------------")
    for idx, result in enumerate(results, 1):
        x = result["x"]
        if has_1d:
            print(
```

Продовження додатку Б

```
                f"{idx:^4}| {result['title'][:28]:<28} | {result['f_evals_rosen']:^7} |
{result['f_evals_1d']:^7} | "
                f"{result['f_evals_total']:^7} | {result['coord_changes']:^7} |
{x[0]:^11.7f} | "
                f"{x[1]:^11.7f} | {fmt_float(result['f'], 7):^11}"
            )
        else:
            print(
                f"{idx:^4}| {result['title'][:28]:<28} | {result['f_evals_total']:^7} |
{result['coord_changes']:^7} | "
                f"{x[0]:^11.7f} | {x[1]:^11.7f} | {fmt_float(result['f'], 7):^11}"
            )
    if has_1d:
        print("----+------------------------------+---------+---------+---------
+---------+-------------+-------------+-------------")
    else:
        print("----+------------------------------+---------+---------+-------------
+-------------+-------------")


def plot_bar_results(title, labels, values, ylabel):
    plt.figure(figsize=(10, 7))
    plt.bar(range(len(values)), values, color="steelblue")
    plt.xticks(range(len(values)), labels, ha="center")
    plt.ylabel(ylabel)
    plt.title(title)
    plt.tight_layout()
    plt.show()


def plot_line_results(title, x_values, y_values, x_label, y_label, x_scale=None,
tick_labels=None):
    plt.figure(figsize=(10, 7))
    plt.plot(x_values, y_values, marker="o")
    if x_scale:
        plt.xscale(x_scale)
    if tick_labels is not None:
        plt.xticks(x_values, tick_labels)
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(title)
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.tight_layout()
    plt.show()


def draw_constraints(ax, constraints):
    for g in constraints:
        kind = getattr(g, "kind", "")
        if kind == "circle":
            t = np.linspace(0, 2 * np.pi, 400)
            ax.plot(
                g.center[0] + g.radius * np.cos(t),
                g.center[1] + g.radius * np.sin(t),
                linestyle="--",
                color="black",
                linewidth=2,
            )
        elif kind == "line":
            x_line = np.linspace(-3, 3, 300)
            y_line = (g.b - g.a1 * x_line) / g.a2
            ax.plot(x_line, y_line, linestyle="--", color="black", linewidth=2)
        elif kind == "nonconvex":
            t = np.linspace(0, 2 * np.pi, 400)
            ax.plot(2.0 * np.cos(t), 2.0 * np.sin(t), linestyle="--", color="black",
linewidth=2)
            hole_r = np.sqrt(0.10)
            ax.plot(1.0 + hole_r * np.cos(t), 1.0 + hole_r * np.sin(t), linestyle="--",
color="black", linewidth=2)
```

Продовження додатку Б

```
def plot_trajectory(title, paths, constraints):
    grid = np.linspace(-3, 3, 200)
    X, Y = np.meshgrid(grid, grid)
    Z = 100 * (X ** 2 - Y) ** 2 + (X - 1) ** 2

    fig, ax = plt.subplots(figsize=(7, 7))
    ax.contour(X, Y, Z, levels=20, cmap="viridis", alpha=0.6)
    ax.contour(X, Y, np.log10(Z + 1e-10), levels=2, cmap="viridis", alpha=0.4)
    draw_constraints(ax, constraints)
    colors = ["crimson", "darkorange", "seagreen"]
    labels = ["Розенброк", "1-D пошук", "Додатковий етап"]
    for idx, path in enumerate(paths):
        points = np.array(path, dtype=float)
        ax.plot(
            points[:, 0],
            points[:, 1],
            marker="o",
            color=colors[idx % len(colors)],
            linewidth=2,
            label=labels[idx] if idx < len(labels) else f"Етап {idx + 1}",
        )
    ax.plot(1.0, 1.0, marker="+", markersize=12, markeredgewidth=2, color="black")
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.set_title(title)
    ax.legend()
    fig.tight_layout()
    plt.show()


def convex_scenarios():
    return [
        ("Опукла область: мінімум всередині", [circular_constraint(2.0)]),
        ("Опукла область: мінімум на межі", [circular_constraint(1.0, center=(-0.5,
0.0))]),
    ]


def region_type_scenarios():
    return [
        ("Опукла область", [circular_constraint(1.0, center=(-0.5, 0.0))]),
        ("Лінійне обмеження x1+x2<=1.5", [linear_constraint(1.0, 1.0, 1.5)]),
        ("Неопукла область", [nonconvex_constraint()]),
    ]


if __name__ == "__main__":
    x0 = np.array([-1.2, 0.0], dtype=float)
    dx0 = np.array([1.95, 0.15], dtype=float)
    a = 4.5
    b = -0.25
    eps = 1e-3
    eps_1d = 1e-3
    sven_alpha = 0.1

    convex_results = [
        run_case(name, constraints, x0, dx0, a=a, b=b, eps=eps)
        for name, constraints in convex_scenarios()
    ]
    print_case_table("1. Основний метод Розенброка на опуклих областях", convex_results)
    plot_bar_results(
        "Опуклі області: кількість обчислень цільової функції",
        [result["title"] for result in convex_results],
        [result["f_evals_total"] for result in convex_results],
        "Nf(x)",
    )
    for result, (_, constraints) in zip(convex_results, convex_scenarios()):
        plot_trajectory(result["title"], result["paths"], constraints)
```

Продовження додатку Б

```
    region_results = [
        run_case(name, constraints, x0, dx0, a=a, b=b, eps=eps)
        for name, constraints in region_type_scenarios()
    ]
    print_case_table("2. Три види областей без допоміжних 1-D функцій", region_results)
    plot_bar_results(
        "Три види областей без допоміжних функцій",
        [result["title"] for result in region_results],
        [result["f_evals_total"] for result in region_results],
        "Nf(x)",
    )
    for result, (_, constraints) in zip(region_results, region_type_scenarios()):
        plot_trajectory(result["title"], result["paths"], constraints)

    line_constraints = [linear_constraint(1.0, 1.0, 1.5)]
    line_results = [
        run_case(
            f"Лінія + 1D пошук: {method}",
            line_constraints,
            x0,
            dx0,
            a=a,
            b=b,
            eps=eps,
            line_method=method,
            eps_1d=eps_1d,
            sven_alpha=sven_alpha,
        )
        for method in ["golden", "dsk"]
    ]
    print_case_table("3. Лінійне обмеження: Розенброк, потім рух уздовж межі",
line_results)
    plot_bar_results(
        "Лінійне обмеження: сумарна кількість обчислень",
        [result["line_method"] for result in line_results],
        [result["f_evals_total"] for result in line_results],
        "Nsum",
    )
    plot_bar_results(
        "Лінійне обмеження: обчислення лише 1-D пошуку",
        [result["line_method"] for result in line_results],
        [result["f_evals_1d"] for result in line_results],
        "N1d",
    )
    for result in line_results:
        plot_trajectory(result["title"], result["paths"], line_constraints)

    eps_1d_values = [1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9, 1e-10]
    eps_results_golden = [
        run_case(
            f"eps_1d = {value:.0e}",
            line_constraints,
            x0,
            dx0,
            a=a,
            b=b,
            eps=eps,
            line_method="golden",
            eps_1d=value,
            sven_alpha=sven_alpha,
        )
        for value in eps_1d_values
    ]
    print_case_table("4a. Точність 1-D пошуку (золотий перетин)", eps_results_golden)
    plot_line_results(
        "Золотий перетин: вплив eps_1d на сумарну кількість обчислень",
        list(range(len(eps_1d_values))),
        [result["f_evals_total"] for result in eps_results_golden],
```

Продовження додатку Б

```
    "eps_1d",
    "Nsum",
    tick_labels=[f"{value:.0e}" for value in eps_1d_values],
)
plot_line_results(
    "Золотий перетин: вплив eps_1d на обчислення 1-D пошуку",
    list(range(len(eps_1d_values))),
    [result["f_evals_1d"] for result in eps_results_golden],
    "eps_1d",
    "N1d",
    tick_labels=[f"{value:.0e}" for value in eps_1d_values],
)

eps_results_dsk = [
    run_case(
        f"eps_1d = {value:.0e}",
        line_constraints,
        x0,
        dx0,
        a=a,
        b=b,
        eps=eps,
        line_method="dsk",
        eps_1d=value,
        sven_alpha=sven_alpha,
    )
    for value in eps_1d_values
]
print_case_table("46. Точність 1-D пошуку (ДСК-Пауелла)", eps_results_dsk)
plot_line_results(
    "ДСК-Пауелла: вплив eps_1d на сумарну кількість обчислень",
    list(range(len(eps_1d_values))),
    [result["f_evals_total"] for result in eps_results_dsk],
    "eps_1d",
    "Nsum",
    tick_labels=[f"{value:.0e}" for value in eps_1d_values],
)
plot_line_results(
    "ДСК-Пауелла: вплив eps_1d на обчислення 1-D пошуку",
    list(range(len(eps_1d_values))),
    [result["f_evals_1d"] for result in eps_results_dsk],
    "eps_1d",
    "N1d",
    tick_labels=[f"{value:.0e}" for value in eps_1d_values],
)

sven_values = [0.001, 0.005, 0.01, 0.05, 0.1, 0.25, 0.5, 1.0]
sven_results = [
    run_case(
        f"a_sven = {value}",
        line_constraints,
        x0,
        dx0,
        a=a,
        b=b,
        eps=eps,
        line_method="golden",
        eps_1d=eps_1d,
        sven_alpha=value,
    )
    for value in sven_values
]
print_case_table("5. Вплив параметра a в алгоритмі Свена", sven_results)
plot_line_results(
    "Вплив параметра a в алгоритмі Свена на сумарну кількість обчислень",
    sven_values,
    [result["f_evals_total"] for result in sven_results],
    "a_sven",
    “Nsum",
```

Продовження додатку Б

```
    x_scale="log",
)
plot_line_results(
    "Вплив параметра a в алгоритмі Свена на обчислення 1-D пошуку",
    sven_values,
    [result["f_evals_1d"] for result in sven_results],
    "a_sven",
    "N1d",
    x_scale="log",
)
```